Provably Retain-Safe Machine Unlearning via Gradient Projection

Anonymous anonymous@example.com
Anonymous Institution

October 25, 2026

Abstract

Machine unlearning must remove the influence of specific training data while preserving model utility—a forget–retain trade-off dual to the stability–plasticity trade-off of continual learning. We exploit this duality and adapt A-GEM gradient projection so that the forget update is projected onto the retain-safe half-space $\{u : \langle u, g_r \rangle \geq 0\}$ at every step. The key design choice is *where* the projection is applied: we project the *post-momentum update actually applied to the weights*, rather than the raw gradient. This makes the resulting per-step retain-safety guarantee hold exactly for the optimiser that is actually run—momentum included—rather than for an idealised momentum-free surrogate. We show that momentum is not incidental: removing it to make the analysis convenient causes the method to stop forgetting on five of six datasets (e.g. forget accuracy 95.9% on MNIST, 73.7% on CIFAR-100), so a guarantee stated for the momentum-free update would describe a method that does not unlearn. We instantiate this in PURGE, which combines (i) the projected update, (ii) a retain-confusion forget objective targeting the model's natural output distribution on retained data, (iii) KD anchoring, and (iv) dual stopping with explicit retain-loss and forget-accuracy budgets. A separation theorem shows that gradient ascent, SCRUB, and SalUn each admit single steps that strictly increase retain loss, whereas PURGE cannot. Across six datasets and 31 class-level forgetting tasks with ResNet-18 (three seeds on the five 10-class datasets; 75 runs), PURGE attains per-dataset mean forget accuracy $\leq 6.2\%$ (STL-10 requires a larger epoch budget to reach 2.8%), retain accuracy $\geq 97.2\%$ (92.2% on CIFAR-100), and membership-inference AUROC within 0.035 of the 0.5 conventionally taken as ideal on all six datasets. That convention, however, does not survive checking. On CIFAR-10 the same attack scores 0.69 against a retrained reference that never saw the forgotten class—higher than against the un-unlearned original—so calibrating against a retrained model rather than against 0.5 reverses the ranking of methods. On PathMNIST the reference is not even constant: it ranges from 0.31 to 0.70 depending on which class is forgotten, so no fixed target is correct and a method can be judged good or bad on one dataset according to which deletion request it receives. Step-level logging confirms the guarantee empirically: the projection is active at 100% of steps, and $\langle u_t, g_r \rangle \geq 0$ holds at every step to floating-point precision. On BatchNorm backbones, forgetting achieved against frozen normalisation statistics is undone by recomputing them, without touching a weight; a variant that keeps the statistics consistent with the weights throughout unlearning keeps forget accuracy at or below the chance-level target under that attack on all ten CIFAR-10 classes and on MNIST, at roughly seven times the compute.

1 Introduction

Trained models increasingly need to demonstrably *forget* specific data—to honour data-deletion requests, revoke expired licences, or remove poisoned samples. Retraining from scratch without the offending data satisfies this exactly, but is computationally prohibitive: a ResNet-18 on CIFAR-10 takes ∼45 minutes on a single A100, and production-scale models take days to weeks. *Machine unlearning* (MU) asks whether we can do better: can we modify an already-trained model so that it behaves as if a specified subset of its training data had never been present, without paying for full retraining?

The difficulty is not removing the data's influence. It is removing that influence *without collateral damage*. A model is a single set of weights in which what we want to erase and what we want to keep are thoroughly entangled; any update that degrades the model's behaviour on the forgotten data also perturbs its behaviour

on everything else. An unlearning method is therefore useful only to the extent that it can control that second effect, and most of the difficulty in the field lies there rather than in the forgetting itself.

A field that already solved the mirror image of this problem. Continual learning (CL) studies models that must acquire new tasks in sequence. Its central obstacle is *catastrophic forgetting*: training on task B overwrites the weights that encoded task A, and accuracy on A collapses. The problem is structurally identical to ours with one sign reversed. Continual learning must change the model to serve a new objective while protecting what it already knows; unlearning must change the model to *destroy* designated knowledge while protecting the rest. In both cases a new objective pulls the weights in some direction, and in both cases the engineering problem is to prevent that pull from damaging knowledge that must survive.

That reframing matters because continual learning has spent a decade developing machinery for exactly this, and the strongest of it is not a penalty term but a *constraint on the update itself*. Averaged Gradient Episodic Memory (A-GEM) [Chaudhry et al., 2019] keeps a small memory of old-task examples and, at every step, computes the gradient g_{old} of the loss on that memory. If the proposed update v would increase old-task loss to first order—detectable as $\langle v, g_{\text{old}} \rangle < 0$—A-GEM replaces v with its projection onto the half-space $\{u : \langle u, g_{\text{old}} \rangle \geq 0\}$, the smallest correction that removes the conflicting component. The old task is protected not because a hyperparameter has been tuned to make damage unlikely, but because a damaging step is not in the set of steps the optimiser can take.

Our insight. This is precisely the guarantee unlearning lacks. Existing methods control retain-set damage through weighted objectives, saliency masks, or alternating updates (§6), all of which make retain damage *unlikely* when their hyperparameters are set well, and none of which make it *impossible*. We therefore treat the retain set as a continual-learning "old task" and the forget objective as the "new task," and adopt the half-space constraint as the update rule for unlearning rather than as a regulariser added to it. The duality between the two problems has been observed before [Kurmanji et al., 2023, Nguyen et al., 2020], and the forget–retain gradient conflict has recently been targeted by a *soft* regulariser [Patel and Qiu, 2025]; what has not been done is to enforce the constraint exactly, at every step, on the vector the optimiser actually applies. Doing so is what turns an empirical tendency into a per-step theorem.

Three challenges, and what is novel in how we meet them. Transplanting the constraint is not mechanical, and each obstacle required a design decision that we believe is novel in this setting.

Challenge 1: a guarantee about the wrong vector is not a guarantee. Practical unlearning uses stochastic gradient descent (SGD) with momentum, so the vector subtracted from the weights is not the current gradient g_{f} but the momentum-accumulated update u_t, which mixes in directions from earlier steps. Projecting g_{f} constrains g_{f} and nothing else: $\langle u_t, g_{\text{r}} \rangle$ can still be negative even when every individual projected gradient was retain-safe (Proposition 1). Two responses are available:

- *Disable momentum*, so that the applied update equals the projected gradient and the analysis becomes exact.
- *Project the accumulated update* u_t rather than the gradient, so that the constraint binds the vector actually applied.

The first is the trap. It yields a clean theorem about a method that does not work: with momentum removed and all else fixed, PURGE stops forgetting on five of six datasets (§5.3), so the guarantee would be exact for an algorithm that fails at its own task. We adopt the second, and regard this—enforcing the constraint on the post-momentum update, so that the theorem and the implementation describe the same object—as the paper's central methodological contribution. Two further ordering decisions (where to clip, and restoring the projected update's magnitude) turn out to be load-bearing for the same reason (§3.2).

Challenge 2: what should the forgotten data be pushed toward? Driving forget-set predictions to the uniform distribution is the common choice, but no trained model produces uniform outputs, so a model driven there remains distinguishable from a retrained one. We instead target the model's own average output on retained data—its *retain-confusion distribution*—which is a behaviour a retrained model can plausibly exhibit (§3.3).

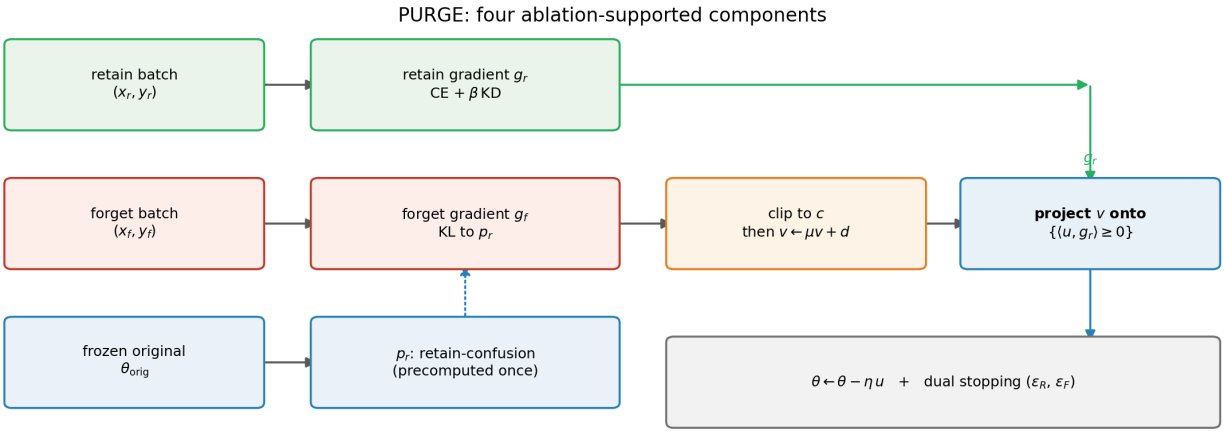


Figure 1: **How PURGE takes a step.** At each iteration the retain batch supplies the constraint gradient g_r and the forget batch supplies the erasure direction. The forget ingredient is clipped, accumulated into the momentum buffer, and only *then* projected onto the retain-safe half-space, so the constraint binds the update u that is actually applied to the weights rather than the raw gradient. The retain gradient is never added to the forget objective—it acts purely as the constraint. Dual stopping halts the procedure when either tolerance is met.

Challenge 3: when should unlearning stop? Erasure that runs too long damages the retained model, and erasure that overshoots the retrained model's behaviour is itself detectable. We derive stopping rules directly from the tolerances of our optimisation problem rather than fixing an epoch count (§3.5).

A fourth issue emerged from the experiments rather than the design. Forget batches contain a single class and would corrupt BatchNorm running statistics, so PURGE freezes them during unlearning—and the weights then learn to forget *relative to* statistics that an attacker can replace. Recomputing them on retained data restores forget accuracy from ∼1% to ∼100% without modifying a single weight. We diagnose this, show that normalisation schemes without running statistics remove it structurally (§5.5), and give a variant that keeps BatchNorm by recomputing its statistics from retained data throughout unlearning, which survives the attack (§5.6).

Figure 1 shows the resulting step, and Figure 2 places the method against the alternatives on both what it achieves and what it guarantees.

Contributions. **(1) Constrained-optimisation framing.** We pose class-level unlearning as constrained optimisation (Eq. 2) with a retain-loss budget, a forget-distribution tolerance, and a trust region, so that each algorithmic component enforces a specific constraint rather than acting as a heuristic (§3 maps each constraint to the mechanism that realises it).

(2) PURGE: a hard per-step constraint, matched to the implementation. Our novelty is not the projection operator, which we take from continual learning, but the claim that it can serve as the *update rule* for unlearning and the demonstration of what that requires: applying it to the post-momentum update, clipping before accumulation, and preserving magnitude under projection. We combine it with the retain-confusion forget objective and dual stopping.

(3) Theory plus an empirical audit of it. Theorem 1 gives an exact per-step retain-safety guarantee for the projected-momentum update, with the update norm bounded by $c/(1-\mu)$. Theorem 2 shows that gradient ascent, SCRUB, and SalUn each admit steps that strictly increase retain loss. We then audit the guarantee at step level on real runs rather than asserting it (§5.4).

(4) A normalisation failure mode, and a fix that keeps BatchNorm. Recalibrating BatchNorm statistics undoes PURGE's forgetting without touching a weight, while leaving the baselines we reproduced unaffected (§5.5). Keeping the statistics consistent with the weights throughout unlearning, and stopping on the forget accuracy the shipped model will actually have, closes it: forget accuracy stays at or below the

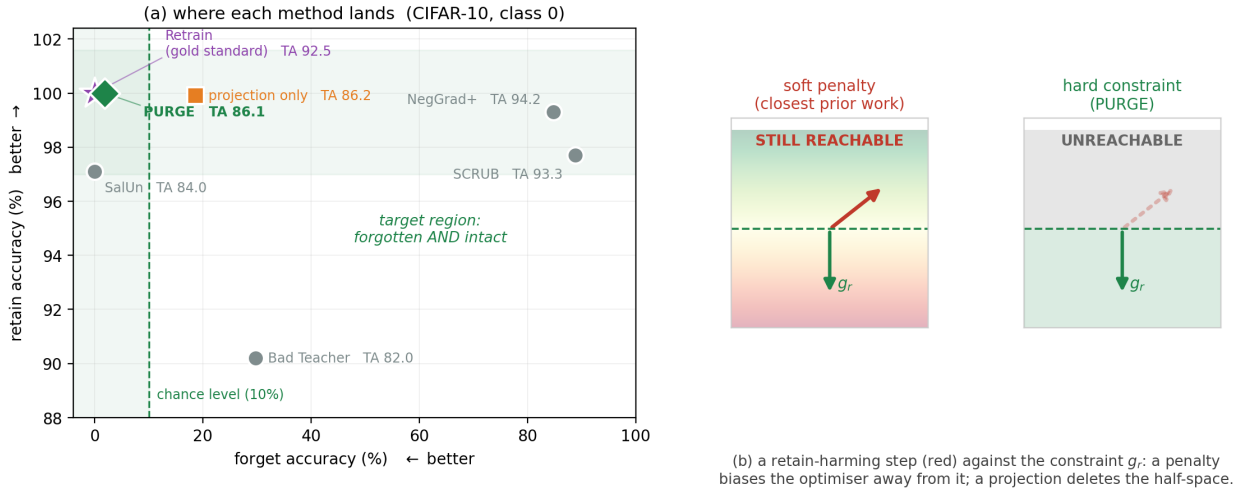


Figure 2: **What PURGE achieves, and what it guarantees. (a)** Measured outcomes on CIFAR-10 class 0 (Table 4): forgetting requires forget accuracy at or below the 10% chance level, and utility requires retain accuracy to stay high. SCRUB and NegGrad+ keep the retained model intact by barely forgetting; Bad Teacher forgets more but damages the model; PURGE is the only method reaching the target region with retain accuracy fully preserved. **(b)** The structural difference, which no accuracy table can show. Against the retain constraint g_r, a soft penalty leaves retain-harming steps (red) inside the feasible set—it makes them costly, not impossible—whereas projecting onto $\{u : \langle u, g_r \rangle \geq 0\}$ removes that half-space entirely (Theorem 2(iv)).

chance-level target under the attack on all ten CIFAR-10 classes and on MNIST (§5.6).

1.1 Related Work and Positioning

Approximate unlearning methods occupy different points on the privacy–utility frontier, and the retain side is where they differ most. **Gradient ascent** [Thudi et al., 2022] maximises forget-set loss with no mechanism protecting retain accuracy, collapsing test accuracy to ∼38% on CIFAR-10. **SalUn** [Fan et al., 2024] confines random-label updates to a saliency mask—effective in practice, but with no formal retain guarantee and no control over how much is forgotten. **SCRUB** [Kurmanji et al., 2023] alternates forget and retain steps balanced by a weight λ, so the balance is tuned rather than enforced; in our reproduction it preserves retain accuracy (97.7%) by barely forgetting at all (forget accuracy 88.8%, Table 4). **NegGrad+** [Thudi et al., 2022] adds a retain-loss term as a weighted sum, again without a guarantee that retain loss is controlled. Closest to us, Patel and Qiu [2025] diagnose the same forget–retain gradient conflict but resolve it with a soft regulariser that rewards positive alignment. A fuller treatment, including exact unlearning and distillation-based methods, appears in §6.

The distinction we draw is structural rather than numerical, so Table 1 compares properties of each method's *definition* —checkable against the published update rule, without relying on our reimplementation of it. A soft penalty biases the optimiser toward retain-safe steps but leaves harmful steps feasible: for any finite penalty weight, a sufficiently large forget-loss reduction still makes a retain-harming step preferable. A projection removes those steps from the feasible set altogether (Theorem 2(iv)). The final row is the one we would emphasise, and the one §5.3 makes concrete: a guarantee proved for a momentum-free surrogate says nothing about a run that uses momentum.

2 Problem Formulation

Let $f_\theta : \mathcal{X} \to \Delta^K$ be a trained K-class classifier with pre-unlearning weights θ_0, mapping an input to a probability vector over K classes. Write $\sigma(\cdot)$ for the softmax and $p_\theta(x) := \sigma(f_\theta(x)) \in \Delta^K$ for the model's

Table 1: **Structural comparison of update rules.** ✓ = holds by construction, ∼ = partial or hyperparameter-dependent, ✗ = not provided. Entries are properties of each method's *definition*, not measured quantities; an empirical comparison against Patel and Qiu [2025] is future work (§7). P&Q = Patel and Qiu [2025]. †PURGE takes an explicit tolerance and honours it across a 29pp range, but only when the stopping rule is evaluated per step: at a 50-batch check granularity, targets from 5% to 30% collapse onto the same operating point (§5, Table 14).

Property	**GA**	**SalUn**	**SCRUB**	**NG+**	**P&Q**	**PURGE**
Addresses forget–retain gradient conflict	✗	✗	∼	∼	✓	✓
Conflict handled as a constraint, not a penalty	✗	✗	✗	✗	✗	✓
Retain-harming steps are unreachable	✗	✗	✗	✗	✗	✓
Per-step retain-safety guarantee	✗	✗	✗	✗	✗	✓
Explicit forgetting tolerance ϵ_F	✗	✗	∼	∼	∼	✓†
Guarantee holds for the optimiser actually run	✗	✗	✗	✗	✗	✓

predicted distribution on input x. The training set splits into the *forget set* $\mathcal{D}_f$, whose influence must be removed, and the *retain set* $\mathcal{D}_r = \mathcal{D}_{\text{train}} \setminus \mathcal{D}_f$, which must be preserved; we write x_f for an input drawn from $\mathcal{D}_f$ and (x, y) for a labelled retain example. $\mathrm{CE}(\cdot,\cdot)$ denotes cross-entropy and $\mathrm{KL}(\cdot\|\cdot)$ the Kullback–Leibler divergence.

We also fix the four quantities used to evaluate unlearning throughout, so that they are available before the algorithm is stated. *Test accuracy* (TA) is measured on the held-out test set and reports how much of the model's general utility survives. *Forget accuracy* (FA) is measured on $\mathcal{D}_f$; because a retrained model has never seen those samples, a fully unlearned model should drive FA down to at most the chance level $100/K\%$. *Retain accuracy* (RA) is measured on $\mathcal{D}_r$ and should not move. Finally, a *membership-inference attack* (MIA) tries to distinguish forget-set training samples from forget-set test samples, and we score it by the area under its receiver-operating-characteristic curve (AUROC); this is conventionally read against an ideal of 0.5, a convention §5.12 shows to be the wrong reference point.

Define the *retain-confusion distribution* p_r as the average predicted distribution of the frozen base model over the retain set:

$$p_r = \frac{1}{|\mathcal{D}_r|} \sum_{x \in \mathcal{D}_r} \sigma(f_{\theta_0}(x)). \tag{1}$$

This captures how the model naturally confuses classes on retained data—a more realistic unlearning target than the uniform distribution, which no trained model ever produces.

We pose unlearning as a constrained optimisation:

$$\begin{aligned} \theta^* = \arg\min_{\theta} \; & \mathcal{L}_{\mathrm{f}}(\theta; \mathcal{D}_f) \\ \text{s.t.} \quad & \mathcal{L}_{\mathrm{r}}(\theta; \mathcal{D}_r) \le \mathcal{L}_{\mathrm{r}}(\theta_0; \mathcal{D}_r) + \epsilon_R, \\ & \mathrm{KL}(p_\theta(x_f) \| p_r) \le \epsilon_F \;\; \forall\, x_f \in \mathcal{D}_f, \\ & \|\theta - \theta_0\| \le R, \end{aligned} \tag{2}$$

where

$$\mathcal{L}_{\mathrm{f}} = \mathbb{E}_{x_f}\big[\mathrm{KL}(p_\theta(x_f) \| p_r)\big], \qquad \mathcal{L}_{\mathrm{r}} = \mathbb{E}_{(x,y)}\big[\mathrm{CE}(f_\theta(x), y)\big] + \beta\, \mathcal{L}_{\mathrm{KD}}.$$

Here $\beta \ge 0$ weights knowledge-distillation anchoring, ϵ_R is the retain-loss tolerance, ϵ_F the forget-distribution tolerance, and R a trust-region radius enforced via early stopping.

This framing makes three things explicit that weighted multi-objective approaches conflate. First, retain safety is a *hard constraint*, not a soft penalty. Second, the choice of p_r (rather than the uniform distribution) determines the forget-side feasible region. Third, the two tolerances (ϵ_R, ϵ_F) are quantities a practitioner sets, not epochs to be tuned.

Table 2: **From the optimisation problem to the algorithm.** Each constraint in Eq. 2 is enforced by one mechanism, and every mechanism in PURGE traces back to one constraint. Nothing in the algorithm is present as a heuristic.

Constraint (Eq. 2)	**Mechanism**	**Enforced**
$\mathcal{L}_{\mathrm{r}}(\theta) \le \mathcal{L}_{\mathrm{r}}(\theta_0) + \epsilon_R$	Projection of the post-momentum update onto $\{\langle u, g_{\mathrm{r}}\rangle \ge 0\}$ (§3.1–3.2)	Every step, by construction (Thm. 1)
Objective $\min \mathcal{L}_{\mathrm{f}}$	`kl_retain`: descend $\mathrm{KL}(p_\theta(x_f)\|p_r)$ (§3.3)	As the descent direction
$\mathrm{KL}(p_\theta(x_f)\|p_r) \le \epsilon_F$	Forget-accuracy stopping rule (§3.5)	On check, as a tolerance
$\|\theta - \theta_0\| \le R$	Retain-loss budget + gradient clipping c (§3.5)	$\|u_t\| \le c/(1-\mu)$ per step

Why this needs a new algorithm. Writing down Eq. 2 is not by itself progress: one could hand it to a generic constrained solver, or relax it into the weighted objective that existing methods already optimise. Both routes lose the property we are after. A Lagrangian relaxation replaces the retain constraint with a penalty $\lambda \mathcal{L}_{\mathrm{r}}$, which is precisely the soft treatment whose feasible set still contains retain-harming steps (Theorem 2(iv)); and a projected-gradient solver applied naively would enforce feasibility on the gradient while the optimiser applies something else (Proposition 1). What the problem *does* buy is a specification: it says exactly which quantity each mechanism must control. PURGE is the algorithm obtained by realising each constraint with the cheapest mechanism that enforces it exactly, at every step, on the vector actually applied. Table 2 states that correspondence, and §3 derives each row.

3 The PURGE Algorithm

PURGE realises each constraint in Eq. 2 through a specific mechanism: projection enforces the retain constraint (§3.1–3.2); the `kl_retain` objective targets the forget constraint (§3.3); dual stopping enforces the budgets (§3.5). Algorithm 1 gives the pseudocode.

3.1 Gradient Projection for Retain Safety

At each step we compute the retain gradient on a retain mini-batch $\mathcal{B}_r \subset \mathcal{D}_r$ and the forget gradient on a forget mini-batch $\mathcal{B}_f \subset \mathcal{D}_f$, both with respect to the model parameters θ:

$$g_{\mathrm{r}} = \nabla_\theta \Big[\tfrac{1}{|\mathcal{B}_r|} \sum_{(x,y)\in\mathcal{B}_r} \mathrm{CE}(f_\theta(x), y) \;+\; \beta\, \mathcal{L}_{\mathrm{KD}}(\theta; \theta_0, \mathcal{B}_r) \Big], \qquad g_{\mathrm{f}} = \nabla_\theta\, \mathcal{L}_{\mathrm{f}}(\theta; \mathcal{B}_f). \tag{3}$$

When a candidate update v satisfies $\langle v, g_{\mathrm{r}}\rangle < 0$—that is, moving along v would *increase* retain loss to first order—we replace it by its Euclidean projection onto the retain-safe half-space:

$$\Pi_{g_{\mathrm{r}}}(v) = v - \frac{\langle v, g_{\mathrm{r}}\rangle}{\|g_{\mathrm{r}}\|^2}\, g_{\mathrm{r}}, \qquad \text{yielding } \langle \Pi_{g_{\mathrm{r}}}(v), g_{\mathrm{r}}\rangle = 0. \tag{4}$$

What is taken from A-GEM, and what is not. The operator in Eq. 4 is A-GEM's, and we claim no novelty for it: the correction $v \mapsto v - \frac{\langle v,g\rangle}{\|g\|^2} g$ is the standard projection onto a half-space, and Chaudhry et al. [2019] introduced its use in continual learning with g the gradient on an episodic memory of past tasks. We also inherit its two attractive properties: it is the *minimum-norm* correction restoring feasibility, so it is the least destructive repair of a conflicting step, and it costs one extra backward pass and one inner product per step.

Algorithm 1 PURGE unlearning (projected-momentum update)

Require: Trained model θ, forget set $\mathcal{D}_f$, retain set $\mathcal{D}_r$; lr η, momentum μ, clip c, weight decay λ_{wd}
Ensure: Unlearned model θ^*

1: $\theta_{\text{orig}} \leftarrow \text{freeze}(\text{copy}(\theta))$; set all BatchNorm layers to `eval` mode
2: $p_r \leftarrow \frac{1}{|\mathcal{D}_r|} \sum_{x \in \mathcal{D}_r} \sigma(\theta_{\text{orig}}(x))$
3: $L_0 \leftarrow \text{CE}(\theta, \mathcal{D}_r)$; budget $\leftarrow \alpha L_0$; $v \leftarrow 0$
4: **for** epoch $= 1$ to T **do**
5: **for** (x_f, y_f) in $\mathcal{D}_f$ **do**
6: Sample (x_r, y_r) from $\mathcal{D}_r$
7: $g_{\text{r}} \leftarrow \nabla_\theta[\text{CE}(f_\theta(x_r), y_r) + \beta \mathcal{L}_{\text{KD}}]$
8: $g_{\text{f}} \leftarrow \nabla_\theta \mathcal{L}_{\text{f}}(\theta; x_f)$
9: $d \leftarrow \text{clip}_c\big(g_{\text{f}} + \lambda_{\text{wd}} \theta\big)$ *// clip BEFORE accumulation*
10: $v \leftarrow \mu v + d$ *// momentum*
11: **if** $\langle v, g_{\text{r}} \rangle < 0$ **then**
12: $u \leftarrow v - \frac{\langle v, g_{\text{r}} \rangle}{\|g_{\text{r}}\|^2} g_{\text{r}}$; $u \leftarrow u \, \|v\| / \|u\|$ *// project the update, rescale*
13: **else**
14: $u \leftarrow v$
15: **end if**
16: $\theta \leftarrow \theta - \eta u$
17: **end for**
18: **if** $\text{FA}(\theta, \mathcal{D}_f) \leq \epsilon_F$ **or** $\text{CE}(\theta, \mathcal{D}_r) >$ budget **then**
19: **break**
20: **end if**
21: **end for**
22: **return** $\theta^* \leftarrow \theta$

What is new is what the operator is asked to do. In A-GEM the projection is a safeguard on a process whose objective already points the right way—the new task is being learned, and the memory gradient only occasionally interferes; in our runs the conflict is not occasional but universal, firing at 100% of steps (§5.4), because erasing a class and preserving the rest are directly opposed rather than incidentally so. The projection is therefore not a correction applied to an optimiser but the optimiser's defining step. That change of role has three consequences, and they are what §3.2 is about:

1. *It must bind the applied vector.* A-GEM projects a gradient; with momentum the applied vector is not the gradient, and projecting the gradient certifies nothing (Proposition 1).

2. *Its magnitude matters.* A-GEM tolerates $\|\Pi_{g_{\text{r}}}(v)\| \leq \|v\|$ because progress on the new task comes mostly from unconflicted steps. When every step conflicts, that contraction compounds and forgetting stalls, so we restore the norm after projecting.

3. *It bounds nothing on the forget side.* The projection guarantees retain safety and only that; on its own it does not unlearn. Our projection-only ablation reaches forget accuracy 18.66% against a 10% chance level (Table 4), which is why PURGE pairs it with a forget objective rather than presenting the projection as a method.

3.2 Which Vector to Project: the Optimiser Matters

PURGE uses SGD with momentum μ, weight decay λ_{wd}, and gradient clipping c. Three ordering choices determine whether the guarantee is real, and all three matter empirically.

(i) Project the update, not the gradient. The vector applied to the weights is u_t, not g_{f}. We therefore momentum-accumulate first and project afterwards (Algorithm 1, lines 10–15), so that $\langle u_t, g_{\text{r}} \rangle \geq 0$ holds for the update that is actually applied. Projecting g_{f} instead leaves $\langle u_t, g_{\text{r}} \rangle$ unconstrained.

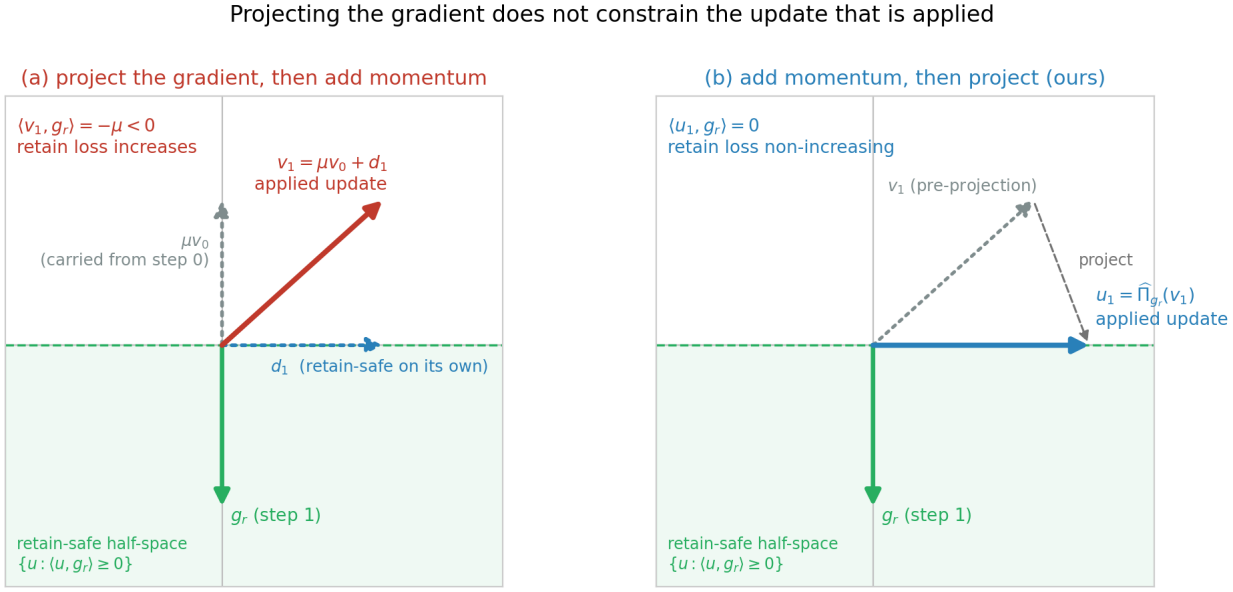


Figure 3: **Why the projection is applied to the update, not the gradient** (Proposition 1). Both panels show step 1, whose constraint is g_r; the retain-safe half-space is shaded. **(a)** Each ingredient is individually retain-safe—d_1 lies on the boundary, and μv_0 was certified against step 0's constraint—yet their sum, the vector actually applied, lies in the harming half-space with $\langle v_1, g_r \rangle = -\mu$. The violation grows with μ. **(b)** Projecting the accumulated update instead returns it to the boundary, so the certificate is issued against the constraint in force where the step is taken.

(ii) Clip before accumulating. Clipping is applied to the per-step ingredient d_t *before* it enters the momentum buffer. This preserves the standard SGD semantics in which momentum accumulates clipped gradients, and yields the bound $\|v_t\| \le c/(1-\mu)$ used in Theorem 1. Clipping the final update instead would cap $\|u_t\| \le c$, destroying the $1/(1-\mu)$ amplification that momentum provides; since raw forget gradients are $\sim$100$\times$ the clip threshold in our runs, clipping is always active, and this ordering is the difference between forgetting and not forgetting.

(iii) Preserve magnitude under projection. Projection is norm-non-increasing: $\|\Pi_{g_r}(v)\| = \|v\| \sin\vartheta$ where ϑ is the angle between v and g_r. When forget and retain gradients are nearly antiparallel, $\sin\vartheta \approx 0$ and the applied step collapses, stalling forgetting for reasons unrelated to retain safety. We therefore rescale the projected update to the pre-projection norm, $u_t = \Pi_{g_r}(v_t)\,\|v_t\|/\|\Pi_{g_r}(v_t)\|$. Because $\langle u, g_r \rangle = 0$ is invariant to positive rescaling, this leaves the guarantee untouched while decoupling forgetting speed from conflict severity. (If $\Pi_{g_r}(v_t) = 0$, i.e. v_t is exactly antiparallel to g_r, we set $u_t = 0$; the guarantee holds trivially.)

3.3 Forget Objective: Retain-Confusion Target

Rather than targeting the uniform distribution $\mathcal{U}(K)$ (which no retrained model produces), the `kl_retain` objective minimises

$$\mathcal{L}_{\text{KL-R}} = \mathrm{KL}(\log \sigma(f_\theta(x_f)) \,\|\, p_r)\,, \tag{5}$$

where p_r (Eq. 1) is the model's natural confusion pattern. The motivation is that no trained model produces uniform outputs, so a model driven toward $\mathcal{U}(K)$ remains separable from a retrained one even at FA = 0%; a forget target drawn from the model's own behaviour on retained data does not have that artefact. Under the previous implementation this was supported empirically, with the uniform target giving MIA AUROC = 0.469 against 0.496 for `kl_retain`. **Re-measured under the projected-momentum optimiser, it is not.** The three forget objectives give, on CIFAR-10 class 0: gradient ascent TA/FA/MIA = 86.4/8.2/0.559, KL-uniform 86.4/3.9/0.526, and `kl_retain` 86.1/2.1/0.528. Both KL variants clearly beat gradient ascent,

which neither forgets as far nor stays as close to the ideal MIA. Between the two, however, the difference is within noise and if anything favours uniform ($|\text{MIA} - 0.5| = 0.026$ against 0.028).

We therefore keep `kl_retain` for its lower forget accuracy at equal test accuracy, and retract the claim that it is distinguishable from the uniform target on MIA grounds. This is consistent with §5.13: p_r is not what a retrained model produces on forget inputs, so an argument resting on p_r being the "natural" target was never on firm ground. What survives is narrower—p_r is a better forget *direction* than uniform, and no more. It is not a claim that p_r is what a retrained model outputs on forget inputs: §5.13 and Appendix F show it is not, and that minimising the divergence to it is harmful.

3.4 Supporting Components

KD-anchored stability. A frozen copy of θ_0 serves as teacher, $\mathcal{L}_{\text{KD}} = T^2\text{KL}(\sigma(f_\theta(x_r)/T)\|\sigma(f_{\theta_0}(x_r)/T))$, anchoring behaviour on $\mathcal{D}_r$ and complementing the projection.

BatchNorm vulnerability, and two fixes. Class-homogeneous forget batches would corrupt BN running statistics, so PURGE freezes them during unlearning. The weights then learn to forget relative to those frozen statistics, and the forgetting does not survive recomputing them on retained data (§5.5). Replacing BN with GroupNorm removes the mechanism by construction (GroupNorm keeps no batch statistics to recalibrate), at a modest accuracy cost. Alternatively, BN-robust PURGE keeps BatchNorm and recomputes its statistics from retained data before every step, so the weights are always trained under the statistics an attacker would install (§5.6).

3.5 Dual Stopping

Two criteria enforce the tolerance budgets of Eq. 2: (1) if retain loss exceeds αL_0, unlearning halts; (2) if FA drops below a target (default $100/K\%$), unlearning is declared complete. An intra-epoch FA check every 50 batches catches rapid drops mid-epoch—necessary because FA is piecewise-constant in θ and can plunge within a single epoch.

4 Theoretical Guarantees

Theorem 1 (Exact per-step retain safety under momentum)**.** *Let the composite retain objective on the sampled retain mini-batch, $\widetilde{\mathcal{L}}_{\text{r}} = \mathcal{L}_{\text{r}} + \beta\mathcal{L}_{\text{KD}}$, be L-smooth, and let $g_{\text{r}} = \nabla_\theta \widetilde{\mathcal{L}}_{\text{r}}(\theta_t)$. Let PURGE run the update of Algorithm 1: $\|d_t\| \le c$ by clipping, $v_t = \mu v_{t-1} + d_t$ with $v_{-1} = 0$ and $\mu \in [0, 1)$, $u_t = \Pi_{g_{\text{r}}}(v_t)\,\|v_t\|/\|\Pi_{g_{\text{r}}}(v_t)\|$ when $\langle v_t, g_{\text{r}}\rangle < 0$ and $u_t = v_t$ otherwise, and $\theta_{t+1} = \theta_t - \eta u_t$. Then, exactly and unconditionally,*

$$\langle u_t, g_{\text{r}}\rangle \ge 0, \qquad \|u_t\| \;=\; \|v_t\| \;\le\; \frac{c}{1-\mu}, \qquad \widetilde{\mathcal{L}}_{\text{r}}(\theta_{t+1}) \;\le\; \widetilde{\mathcal{L}}_{\text{r}}(\theta_t) + \tfrac{1}{2}L\eta^2\Big(\frac{c}{1-\mu}\Big)^2. \tag{6}$$

Proof sketch. The first claim is immediate: if $\langle v_t, g_{\text{r}}\rangle \ge 0$ then $u_t = v_t$; otherwise $\langle \Pi_{g_{\text{r}}}(v_t), g_{\text{r}}\rangle = 0$ identically, and rescaling by a positive scalar preserves this. For the second, $\Pi_{g_{\text{r}}}$ is the orthogonal projection onto $\{u : \langle u, g_{\text{r}}\rangle = 0\}$, so the rescaling restores $\|u_t\| = \|v_t\|$; unrolling the buffer gives $v_t = \sum_{k=0}^{t} \mu^{t-k} d_k$, hence $\|v_t\| \le c\sum_{j=0}^{t}\mu^j < c/(1-\mu)$. The third follows from L-smoothness, $\widetilde{\mathcal{L}}_{\text{r}}(\theta_{t+1}) \le \widetilde{\mathcal{L}}_{\text{r}}(\theta_t) - \eta\langle u_t, g_{\text{r}}\rangle + \frac{1}{2}L\eta^2\|u_t\|^2$, where the first-order term is non-positive by claim one and $\|u_t\|$ is bounded by claim two. The full proof, together with the three supporting lemmas, is given in Appendix B (Theorem 3). □

Three points deserve emphasis. First, the first-order change in the sampled retain loss is non-positive; the only increase is the $O(\eta^2)$ smoothness term. Second, and unlike guarantees stated for momentum-free surrogates, this *matches the implemented optimiser*: the analysed u_t is the vector the code subtracts from the weights, with momentum enabled. Third, the bound is stated for the sampled mini-batch retain objective—the quantity PURGE projects against—not the full-population retain loss; we return to this in §7.

Theorem 2 (Per-step separation)**.** *For any model θ_0 with retain gradient g_{r}: **(i)** GA's update $+\eta g_{\text{f}}$ strictly increases retain loss to first order whenever $\langle g_{\text{f}}, g_{\text{r}}\rangle > 0$; **(ii)** SCRUB's $-g_{\text{f}} + \lambda g_{\text{r}}$ can have $\langle \cdot, g_{\text{r}}\rangle < 0$ for any fixed λ; **(iii)** SalUn's masked random-label gradient has unconstrained sign with respect to g_{r}; **(iv)** a soft*

conflict regulariser that adds $\rho\,\phi(\langle g, g_r\rangle)$ with ϕ decreasing—the approach of the closest prior work [Patel and Qiu, 2025]—shifts the optimum toward positive alignment but leaves the feasible set unrestricted: for any finite ρ, a sufficiently large forget-loss reduction still makes a step with $\langle g, g_r\rangle < 0$ preferable. Only PURGE *guarantees $\langle u_t, g_r\rangle \geq 0$ unconditionally, because only a projection makes such steps unreachable rather than merely costly.*

5 Experiments

5.1 Setup

Datasets. We evaluate on six image-classification benchmarks chosen to vary the three factors that plausibly govern how hard class-level unlearning is: the number of classes, the number of examples per class, and the semantic distance between the forgotten class and the retained ones.

CIFAR-10 [Krizhevsky, 2009] is the standard unlearning benchmark and serves as our reference setting. MNIST [LeCun et al., 1998] and SVHN [Netzer et al., 2011] share its ten-class structure but differ in difficulty and in how strongly classes overlap, which lets us separate the method's behaviour from the particular geometry of CIFAR-10. CIFAR-100 [Krizhevsky, 2009] raises the class count tenfold, lowering the chance level to 1% and shrinking each forget set to 500 images—a regime where "forgotten" is a far stricter requirement. STL-10 [Coates et al., 2011] isolates data scarcity in the other direction: at 96×96 resolution with only 500 training images per class, the forget set yields roughly 60 optimiser steps, which stresses any method that must make progress under a per-step constraint. PathMNIST [Yang et al., 2023] moves off natural images entirely to histopathology tissue types, where class boundaries are less separable and where deletion requests are a realistic rather than hypothetical requirement.

Forgetting a single class can be unrepresentative—classes differ substantially in difficulty—so wherever cost permits we forget *every* class in turn: all ten for MNIST and all nine for PathMNIST, with per-class results reported in Appendix D rather than only in aggregate. For CIFAR-100 we sample one class per tens group (classes 10–90) to cover the label space at tractable cost, and for CIFAR-10, SVHN, and STL-10 we forget class 0 across three seeds to quantify run-to-run variance. This gives 31 forgetting tasks and 75 unlearning runs in total.

Model, baselines, metrics. All experiments use ResNet-18 [He et al., 2016] (11.7M parameters). We compare against Retrain (gold standard), SalUn [Fan et al., 2024], SCRUB [Kurmanji et al., 2023], Bad Teacher [Chundawat et al., 2023], and NegGrad+ [Thudi et al., 2022], all re-run on our stack with a *unified* MIA protocol, so that every number is produced by the same attack implementation rather than quoted from each method's own paper. We report the four metrics defined in §2: TA, FA, RA, and MIA AUROC. Two reminders about FA are worth repeating here: the target is the chance level $100/K\%$ rather than zero, and a retrained reference in fact reaches ≈0% because it routes forget-class inputs to surviving classes.

Hyperparameters. CIFAR-10: $\eta = 5{\times}10^{-4}$, $\mu = 0.9$, $\beta = 2.0$, $c = 1.0$, $\alpha = 5.0$, $T_{\mathrm{KD}} = 4.0$. Other datasets use $\eta = 2{\times}10^{-4}$, $\beta \in [3, 5]$. All configs are released as YAML.

5.2 Cross-Dataset Results

Table 3 reports the main results. At the shared epoch budget PURGE drives FA to or below the chance level on four of six datasets and holds RA $\geq$ 97.2% everywhere except CIFAR-100 (92.2%); at the extended budget it reaches chance on all six. Every one of the ten MNIST classes and all nine PathMNIST classes individually land below their respective chance levels. The two shortfalls in the upper block—STL-10 and CIFAR-100—are budget-limited rather than method-limited, and §5.2 separates the two explanations.

The per-dataset mean MIA AUROC stays within 0.035 of 0.5 on all six datasets. We report that because it is the conventional summary, but §5.12 shows 0.5 is not the right reference point for this protocol—and that no fixed reference point is—so it should not be read as evidence of successful unlearning on its own.

The MIA figure above is a per-dataset *mean*, and on PathMNIST that mean conceals a defect: two individual classes sit at 0.298 and 0.388, well *below* 0.5 rather than above it (Appendix D). We report this rather than let the aggregate absorb it, and we return to it in §5.13.

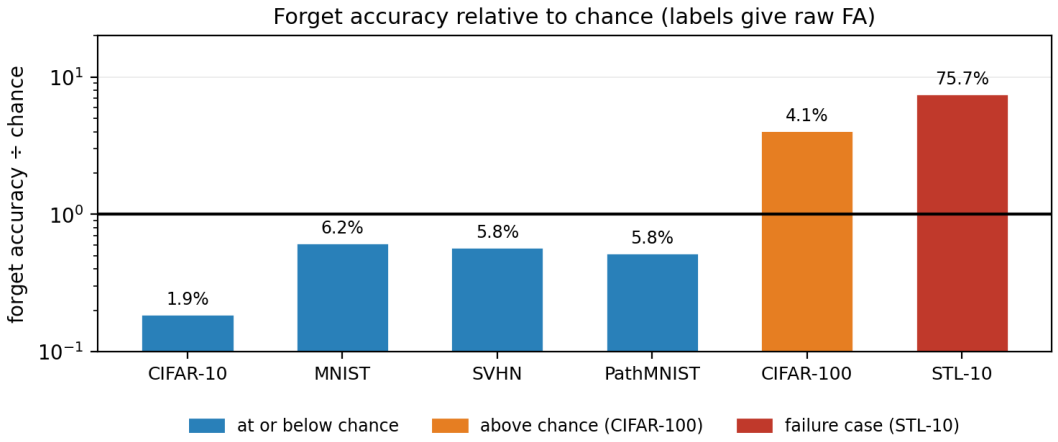


Figure 4: **Forget accuracy relative to the chance level.** Raw forget accuracy is not comparable across datasets: CIFAR-100's 4.1% is *above* its 1% chance level while MNIST's 6.2% is *below* its 10% one. Dividing by $100/K$ puts "at chance" at 1.0 everywhere. Four datasets sit below chance, CIFAR-100 above it despite a ∼95pp reduction, and STL-10 above it at the shared epoch budget, though not at four times that budget (§5.2).

The two shortfalls are budget-limited, not method-limited. Table 3 uses one epoch budget for every dataset, which is the right protocol for a cross-dataset comparison but penalises datasets with few images per class, since the binding resource is update steps rather than examples. STL-10 supplies only 4 forget mini-batches per epoch, so the shared 15-epoch budget buys ∼60 steps against ∼600 for CIFAR-10. Raising STL-10 to 60 epochs takes forget accuracy from 75.67% to **2.80**%, below its 10% chance level, at RA = 98.91% and MIA = 0.491; the untargeted 60-epoch run reaches 9.20% without a tuned tolerance, so the result is not knife-edge. Test accuracy moves from 92.84% to 85.21%, which is not collateral damage but the arithmetic of having actually forgotten: TA is measured over all K classes, so removing one of ten costs a tenth of it, and $85.15 = 94.61 \times \frac{9}{10}$ leaves the retained classes at 94.61%. The same operation on CIFAR-10 gives $86.13 = 95.70 \times \frac{9}{10}$, so under the longer budget STL-10 behaves like the datasets that already worked rather than like a failure case.

CIFAR-100 responds likewise, and more informatively. Re-running all nine tens-group classes at 60 epochs takes the mean from 4.07% to $\mathbf{1.27 \pm 0.37}$% against a 1% chance level—so the chance level now lies within one standard deviation—while the spread across classes collapses from ±5.51 to ±0.37. The earlier variance was therefore not an intrinsic property of the 100-class setting but a symptom of some classes being halted before convergence: the outlier that failed to converge within 20 epochs (FA 17.8%) reaches 2.0% given 60. Retain accuracy costs 0.87pp for this (92.19 → 91.32).

Both re-runs use the same seeds and classes as the shared-budget rows, so we report them as a second block of Table 3 rather than substituting them into the first. The distinction is worth preserving: at a common epoch budget the method reaches chance on four of six datasets, and at a budget scaled to the data it reaches chance on all six. Which is the honest headline depends on whether a practitioner will spend four times the compute on small or many-class datasets, so we report both and label them.

The projection alone does not unlearn. The *projection-only* row is the direct projected-gradient baseline: A-GEM projection applied to plain gradient ascent, with the KD anchor and the retain-confusion target removed. It preserves the retain set almost perfectly (RA = 99.90%), which is what the guarantee promises and all that it promises—but it stops at FA = 18.66%, above the 10% chance level, and halts on the retain-loss budget rather than on the forget target. Read together with the equal-compute ablation (Table 12), where *removing* the projection costs 47.4pp of retain accuracy, this delineates the division of labour precisely: the projection supplies retain safety and nothing else, while the `kl_retain` objective is what actually drives forget accuracy to chance. Neither component is sufficient alone, which is the substantive content of the ablation.

Table 3: **Cross-dataset results.** Mean ± std over classes × seeds (3 seeds for the 10-class datasets; single seed per class for CIFAR-100). #T = number of forgetting tasks. Chance = $100/K\%$ is the level a fully unlearned model must reach. The upper block holds the epoch budget fixed across datasets, which is the right control for a cross-dataset comparison but penalises datasets with few images per class, since the binding resource is optimiser steps rather than epochs. The lower block re-runs the two datasets that fall short at four times that budget, on the same seeds and classes; *marks the shared-budget STL-10 row it supersedes (§5.2).

Dataset	#T	Chance	TA	FA↓	RA	MIA
CIFAR-10	1	10.0%	86.13 ± 0.02	$\mathbf{1.87} \pm 0.25$	100.00 ± 0.00	0.529 ± 0.001
MNIST	10	10.0%	90.00 ± 0.64	$\mathbf{6.17} \pm 2.81$	99.49 ± 0.04	0.497 ± 0.009
SVHN	1	10.0%	89.69 ± 0.02	$\mathbf{5.75} \pm 0.17$	97.26 ± 0.01	0.525 ± 0.001
STL-10*	1	10.0%	92.84 ± 0.02	75.67 ± 1.70	99.38 ± 0.10	0.466 ± 0.005
PathMNIST	9	11.1%	80.96 ± 5.39	$\mathbf{5.82} \pm 3.42$	98.88 ± 0.31	0.483 ± 0.084
CIFAR-100	9	1.0%	76.17 ± 0.86	4.07 ± 5.51	92.19 ± 0.84	0.500 ± 0.037
Extended budget (60 epochs; same seeds, classes and tolerances)						
STL-10	1	10.0%	85.15 ± 0.05	$\mathbf{1.73} \pm 0.77$	98.87 ± 0.04	0.491 ± 0.007
CIFAR-100	9	1.0%	75.50 ± 0.64	$\mathbf{1.27} \pm 0.37$	91.32 ± 0.88	0.494 ± 0.038

Table 4: **CIFAR-10 class-0 forgetting.** PURGE is a 3-seed mean ± std. Methods with FA > 80% attain MIA ≈ 0.5 trivially, by barely changing the model. †Our reproduction; ‡our reproduction with unified MIA. §A-GEM projection applied to plain gradient ascent, with PURGE's KD anchor and retain-confusion target removed: the direct projected-gradient baseline, isolating what the projection alone achieves.

Method	TA	FA↓	RA	MIA
Retrain (gold standard)	92.5	0.0	100.0	0.500
SalUn†	84.0	0.0	97.1	0.497
Bad Teacher‡	82.0	29.7	90.2	0.499
SCRUB‡	93.3	88.8	97.7	0.515
NegGrad+‡	94.2	84.8	99.3	0.525
Projection-only§	86.23	18.66	99.90	0.511
PURGE	**86.13** ± .02	**1.87** ± .25	**100.00** ± .00	**0.529** ± .001

CIFAR-100: scaling to 100 classes. Table 5 gives per-class results (base model TA = 78.5%, chance = 1%). PURGE reduces FA from ∼99% to a mean of 4.07% (median 1.60%) while holding RA at 92.19%, with MIA averaging 0.500. Eight of nine classes reach FA ≤ 6.8%; class 60 is an outlier at 17.8% that does not converge within the 20-epoch budget, and it alone accounts for the gap between the mean and the median (excluding it, mean FA is 2.35%). Unlike the 10-class datasets, CIFAR-100 does not reach its 1% chance level, so we report it as a ∼95pp reduction rather than as forgetting to chance.

5.3 Momentum Is Not Incidental: Why the Guarantee Must Match the Optimiser

The most direct route to an exact per-step guarantee is to remove momentum, so that the applied update *is* the projected gradient. Table 6 shows what that costs. With $\mu = 0$ and all else fixed, PURGE fails to forget on five of six datasets: FA remains at 95.9% on MNIST, 99.7% on STL-10, 73.7% on CIFAR-100, and 61.9% on SVHN. Retain accuracy stays high throughout—but only because the model barely moves; high RA together with high FA is the signature of a method that preserves retained knowledge by declining to unlearn.

This is the empirical content of our central methodological claim. A per-step guarantee stated for the

Table 5: **CIFAR-100 class-level forgetting** (single seed per class, chance = 1%). *Class 60 does not converge within the 20-epoch budget.

Forget class	TA (%)	FA (%)↓	RA (%)	MIA
10	77.23	4.20	92.84	0.485
20	76.69	1.60	92.80	0.485
30	75.18	1.40	91.03	0.486
40	77.19	6.80	93.14	0.506
50	76.25	0.60	92.31	0.562
60*	76.76	17.80	92.86	0.445
70	75.38	1.60	90.89	0.482
80	74.95	1.00	91.51	0.553
90	75.87	1.60	92.32	0.498
Mean	**76.17**	**4.07**	**92.19**	**0.500**
Median	**76.25**	**1.60**	**92.31**	**0.486**

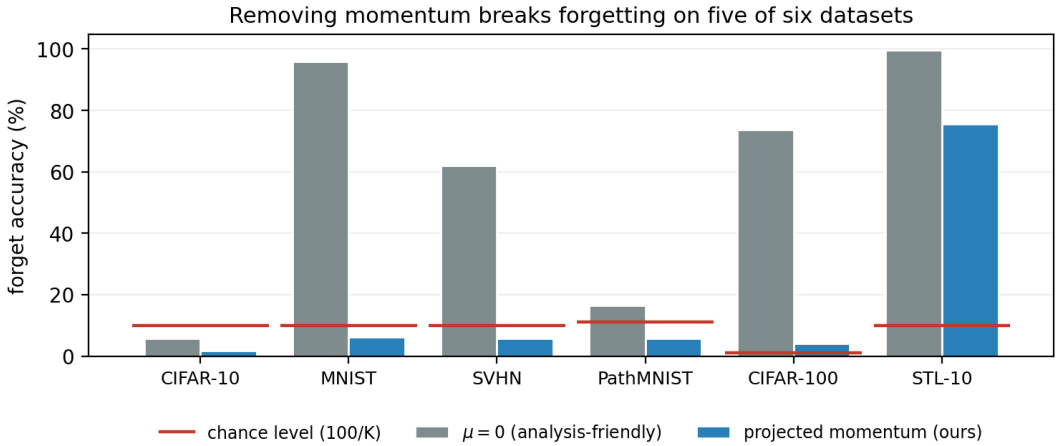


Figure 5: **Momentum is not incidental.** Forget accuracy with momentum removed ($\mu = 0$, grey) against the projected-momentum update (blue), with each dataset's chance level in red. The momentum-free configuration—the one for which an exact guarantee is easiest to state—fails to forget on five of six datasets. Retain accuracy stays high throughout (Table 6), because the model barely moves.

momentum-free update would be exact and implementation-matched, yet would describe a method that does not unlearn. Projecting the post-momentum update instead keeps the guarantee exact *and* recovers forgetting on every dataset where it is attainable.

Is the dependence ours, or the field's? A natural objection is that this ablation only tells us something about PURGE. If every gradient-based unlearning rule collapses without momentum, the finding is about optimisers and not about our design; if only ours does, the hard constraint is fragile. We therefore repeated the ablation across four update rules on the two datasets where Table 6 shows the largest effect, at *equal compute* with stopping disabled so that the momentum change is not confounded with step count (Table 7).

Every rule degrades without momentum, including plain gradient ascent, which uses no projection at all—so the dependence is not an artefact of operating under a hard per-step constraint. But the rules do not degrade equally, and the grouping is informative. The two KL-based objectives lose 70–79pp of forget accuracy on average, the two gradient-ascent objectives 12–15pp: a factor of 5.6. Meanwhile the like-for-like comparison that isolates the projection—gradient ascent with it (+14.8) against gradient ascent without it (+12.0)—differs by only 2.8pp.

Table 6: **Momentum ablation.** Removing momentum to simplify the analysis breaks forgetting on five of six datasets. High RA with high FA indicates the model barely changed. Same configs, seeds, and stopping rules throughout.

		$\mu = 0$ (analysis-friendly)		Projected momentum (ours)	
Dataset	**Chance**	**FA↓**	**RA**	**FA↓**	**RA**
CIFAR-10	10.0%	5.8	100.0	1.9	100.0
MNIST	10.0%	95.9	99.5	6.2	99.5
SVHN	10.0%	61.9	97.4	5.8	97.3
STL-10	10.0%	99.7	99.4	75.7	99.4
PathMNIST	11.1%	16.4	99.0	5.8	98.9
CIFAR-100	1.0%	73.7	94.1	4.1	92.2

Table 7: **Where the momentum dependence comes from** (equal compute, 12 epochs on MNIST and 15 on SVHN, dual stopping disabled). Δ is the increase in forget accuracy when momentum is removed, so larger is more dependent. Grouping by forget objective separates the rows cleanly; grouping by whether the projection is used does not.

Update rule	Forget objective	Projection	MNIST Δ	SVHN Δ	mean
PURGE	KL $\to p_r$	yes	+95.3	+62.4	**+78.8**
`kl_uniform`	KL $\to$ uniform	yes	+84.9	+56.3	**+70.6**
projection only	gradient ascent	yes	+16.5	+13.2	+14.8
gradient ascent	gradient ascent	no	+11.5	+12.5	+12.0

The momentum dependence therefore belongs to the *forget objective*, not to the projection. The reason is visible in the objective itself: $\mathrm{KL}(p_\theta(x_f)\|p_r)$ is bounded and its gradient vanishes as $p_\theta \to p_r$, so the per-step signal is small and the $1/(1-\mu) = 10\times$ amplification that momentum supplies (Lemma 2) is what makes progress possible at all. Gradient ascent maximises an unbounded loss and needs no such amplification, which is also why it damages the retained model.

This sharpens rather than weakens the section's claim. The point was never that momentum is uniquely important to PURGE; it is that the guarantee must be stated for the update the optimiser applies, and any method whose objective requires momentum inherits that requirement. A momentum-free analysis is convenient for exactly the objectives that cannot afford to be momentum-free.

5.4 Auditing the Guarantee at Step Level

We log every gradient step of a CIFAR-10 class-0 run (120 steps) and check Theorem 1 directly rather than assuming it.

- **The constraint is active, not vacuous.** All 120/120 steps have $\langle v_t, g_r\rangle < 0$, so the projection fires at every step. Projection here is not an occasional correction: without it, every step would increase retain loss to first order.
- **The guarantee holds.** After projection, $\langle u_t, g_r\rangle \geq 0$ at every step to floating-point precision. The largest relative violation is $\langle u_t, g_r\rangle/(\|u_t\|\|g_r\|) = -7.4 \times 10^{-7}$, against a float32 accumulation scale of $\sim 4 \times 10^{-4}$ for sums over 11.2M parameters; the mean relative alignment is positive ($+2.2 \times 10^{-8}$). The implied worst-case first-order retain increase is 3.1×10^{-7}, three orders of magnitude below the observed per-step retain-loss fluctuation (1.8×10^{-4}). We therefore state the guarantee as holding to float32 precision, and recommend float64 accumulation for the inner products in reference implementations.
- **The norm bound is respected.** $\|u_t\|$ reaches a maximum of 9.16 against the theoretical bound $c/(1-\mu) = 10$, with mean 7.35—close to but never exceeding the bound, confirming both that the bound is tight and that momentum's amplification is being realised.

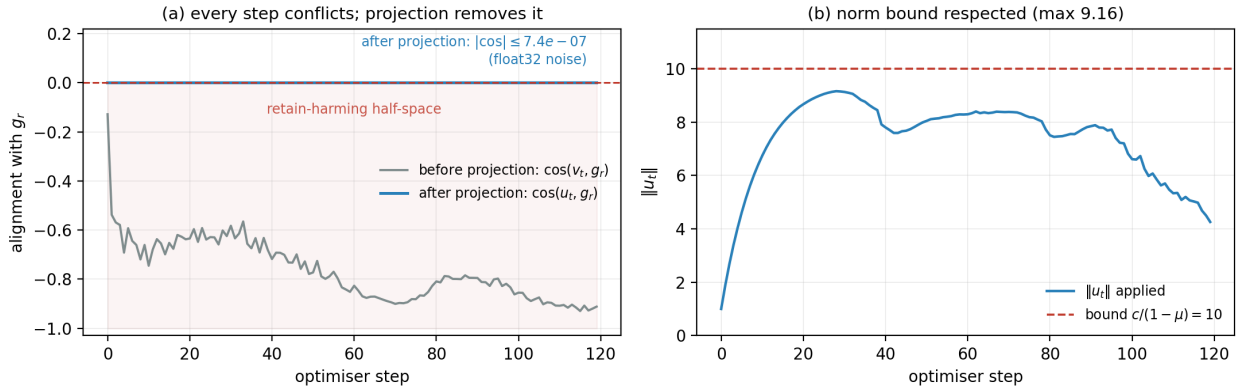


Figure 6: **Theorem 1 audited on a real run** (CIFAR-10 class 0, 120 steps). **(a)** Alignment with the retain gradient, normalised to $\cos(\cdot, g_r)$. Before projection every step lies in the retain-harming half-space (mean -0.77); after projection the alignment is indistinguishable from zero ($|\cos| \leq 7.4 \times 10^{-7}$, the float32 accumulation scale). **(b)** The applied update never exceeds the bound $c/(1-\mu) = 10$ of Eq. 6, peaking at 9.16—tight, but respected.

Table 8: **PURGE with a GroupNorm backbone** (3-seed means). GroupNorm removes the BN-recalibration vulnerability structurally. *STL-10 is under-trained under both backbones.

Dataset	TA	FA↓	RA	MIA
CIFAR-10	84.16	3.44	99.23	0.478
MNIST	90.17	5.84	99.31	0.500
SVHN	87.99	3.46	95.49	0.581
STL-10*	83.92	97.60	92.95	0.552
PathMNIST	72.00	2.71	95.97	0.583

- **Retain loss is controlled.** The mean per-step change in mini-batch retain CE is -7.9×10^{-5} (i.e. decreasing), with a maximum single-step increase of $+1.05 \times 10^{-3}$, consistent with the $O(\eta^2)$ term of Eq. 6.

5.5 BatchNorm Recalibration and the GroupNorm Backbone

Table 8 reports PURGE with a GroupNorm backbone across datasets. GroupNorm keeps forgetting intact on four of five datasets at a modest TA cost (e.g. CIFAR-10 84.16% vs 86.13%), while removing the BN-recalibration mechanism by construction, since there are no batch statistics to recalibrate. STL-10 is under-trained under GroupNorm as it is under BN, at the shared budget (§5.2).

Table 9 reports the diagnostic itself. The operator is `torch.optim.swa_utils.update_bn` applied to the retain set: it resets every BatchNorm buffer, sets the momentum to `None` so that PyTorch accumulates a cumulative rather than exponential average, and runs forward passes with no backward pass and no weight update. The resulting statistics are therefore the exact retain-set statistics, independent of the pre-attack buffers and idempotent in the data. We verify after every run that all learnable parameters are bit-identical, so nothing but the buffers can have changed. Then $\Delta F = \text{post} - \text{pre}$ separates weight-level forgetting ($\Delta F \approx 0$) from forgetting mediated by batch statistics ($\Delta F \gg 0$). We also report a weaker variant—forward passes in `train` mode under the default exponential average, which is what an earlier version of this work used—and the two agree to within 1.2pp on every row, and to 0.02pp on the PURGE row that carries the finding, so the choice of estimator is not what drives the result. All accuracies here are measured under deterministic (test-time) transforms; leaving training augmentation active perturbs individual rows by several points.

Table 9: **BatchNorm recalibration** (CIFAR-10, class 0), using `update_bn` on the retain set: BatchNorm buffers are reset and recomputed, no weight is touched. Every baseline and the un-unlearned base model are unaffected ($\Delta F \leq 0$); PURGE with a BatchNorm backbone loses essentially all of its forgetting. The two backbones without running statistics are outside the operator's reach by construction. NegGrad+ is omitted: no checkpoint for it was available.

Configuration	pre-F	post-F	$\boldsymbol{\Delta F}$	pre-TA	post-TA
Base model (no unlearning)	100.0	99.8	−0.2	95.5	92.9
SalUn	0.0	0.0	+0.0	84.7	84.4
Bad Teacher	28.7	28.4	−0.3	81.5	84.0
SCRUB	89.3	85.7	−3.6	93.5	93.2
PURGE (BatchNorm, frozen)	1.4	99.7	**+98.4**	86.1	92.9
PURGE (GroupNorm)	4.9	4.9	**0.0**	84.2	84.2
PURGE (ViT-B/16, LayerNorm)	9.6	9.6	**0.0**	87.0	87.0

The vulnerability is ours, not the field's—in our reproductions. The baseline rows matter more than PURGE's. Every one of them is unaffected by recalibration—the largest movement is SCRUB's −3.6pp, in the direction of *more* forgetting—and so is the base model, which is the control that matters: recalibrating a normally trained network changes nothing, because its statistics already agree with its weights. Only PURGE carries a mismatch. $\Delta F = +98.4$pp means the attack does not merely degrade the unlearning but *undoes* it: forget accuracy returns to 99.7% and test accuracy to 92.9%, both essentially the pre-unlearning model, without a single weight being modified. On a BatchNorm backbone, what PURGE produces is not an unlearned model but an unlearned model's statistics.

We qualify the first half of that heading deliberately. Concurrent work diagnosing this artifact reports large recoveries for SalUn and Bad Teacher, where we measure none, using its own reproductions of those methods. We ran the identical operator, so the disagreement is not in the attack; it is in how the unlearned checkpoints were produced, and specifically in whether a method let its normalisation statistics move during unlearning. Our claim is therefore that the vulnerability is PURGE-specific *among the checkpoints we trained*, not that these baselines are immune in general—a distinction we cannot settle without their checkpoints, and one a reader should keep when comparing the two sets of numbers.

We attribute it, tentatively, to the interaction between our own design choice and BatchNorm. PURGE freezes the normalisation statistics during unlearning so that class-homogeneous forget batches cannot corrupt them; the weights therefore adapt against *fixed* statistics, and the resulting decision boundary is valid only for those statistics. Methods that let the statistics move during unlearning never build that dependence. Freezing BatchNorm is thus necessary but not sufficient: it prevents the statistics from manufacturing apparent forgetting, and simultaneously makes the weights depend on them. Section 5.6 puts this attribution to an interventional test: keeping the statistics consistent with the weights throughout unlearning removes the vulnerability.

The GroupNorm backbone removes the mechanism by construction—there are no batch statistics to recalibrate—at a cost of 2.0pp of test accuracy (Table 8). The ViT-B/16 backbone (§5.7) does the same for a different reason: LayerNorm normalises per example at inference and keeps no running statistics either. Two architectures with unrelated designs are both outside the operator's reach, and both unlearn properly, which locates the defect in the normalisation scheme rather than in gradient projection.

We state the limit of that plainly: neither backbone *resists* the attack, because the attack is undefined on them. This is a design change, not demonstrated robustness, and a method that must avoid BatchNorm to be safe is constrained in a way the guarantee itself does not capture. What the two rows do establish is that the defect is separable from PURGE's mechanism—it is a property of freezing statistics that a normalisation layer keeps, and it disappears wherever there are no such statistics. The next subsection removes the constraint itself: it keeps BatchNorm, and makes PURGE robust to the attack rather than outside its reach.

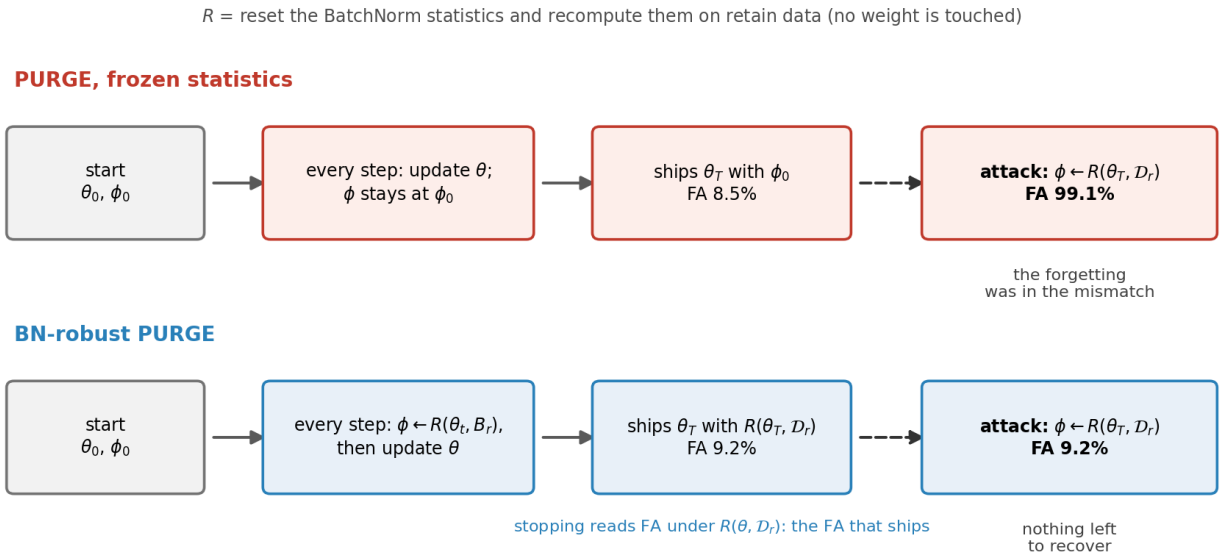


Figure 7: **What BN-robust PURGE changes.** R resets the BatchNorm statistics and recomputes them on retain data without touching a weight. With frozen statistics the forgetting lives in the mismatch between θ_T and ϕ_0, and the attack removes it; BN-robust ships statistics consistent with its weights, so the attack has nothing to recover. Forget accuracies are CIFAR-10 class 0, means over three seeds (Table 10).

5.6 Keeping BatchNorm: Recalibrating During Unlearning

The diagnosis above points at its own remedy: if the weights learn to forget against statistics that are allowed to go stale, the statistics should never be stale. BN-robust PURGE makes three changes to Algorithm 1 (Figure 7). It leaves the update rule, and so Theorem 1, untouched.

1. *Track the weights.* Before every step the BatchNorm statistics are recomputed, $\phi \leftarrow R(\theta, B_r)$, with the attacker's own operator R (`update_bn`) on a fresh sample B_r of 512 retain images. The step's forward and backward passes then run under statistics consistent with the current weights. Forget batches still never touch the statistics: this keeps the half of freezing that was right and drops the half that was wrong.

2. *Ship consistent statistics.* After the last step, $\phi \leftarrow R(\theta_T, \mathcal{D}_r)$ on the whole retain set under test-time transforms—exactly what the attacker computes.

3. *Stop on the forget accuracy that ships.* The stopping rule of §3.5 reads FA under $R(\theta, \mathcal{D}_r)$ with test-time transforms, not under the in-loop statistics. If the target is not met within the epoch budget, the epoch with the lowest such FA inside the retain budget is shipped instead of the last one.

Because R is idempotent, change 2 makes the attack a no-op on the shipped model by construction, so $\Delta F = 0$ is not by itself evidence here. The question this variant must answer is whether FA is still low once ϕ cannot be stale. Two of the tests below are not no-ops by construction: the exponential-average operator, and recalibration on data other than the retain set.

Change 3 is not cosmetic. Without it, the FA that triggers stopping is read under the in-loop statistics—a few hundred augmented images, one step behind—on augmented forget images. On CIFAR-10 that reading falls short of the shipped FA by between -1.0 and $+9.0$pp depending on the class, and runs stopped at "FA $\leq$ 10%" shipped at 7.2–16.7% (Figure 8). With change 3 the shipped FA meets the target, and the seed-to-seed spread on class 0 falls from 1.90 to 0.08pp (Table 10). The stopping check need not use the whole retain set. Computing it on 10,240 images reproduced every stopping decision of the full check on all three seeds and cut wall-clock by 16%; the shipped statistics still use the full set.

These experiments were run after the checkpoints behind Table 9 became unavailable. They use an earlier CIFAR-10 base (test accuracy 93.97%, against 95.5% there), a GroupNorm base retrained for 100 rather

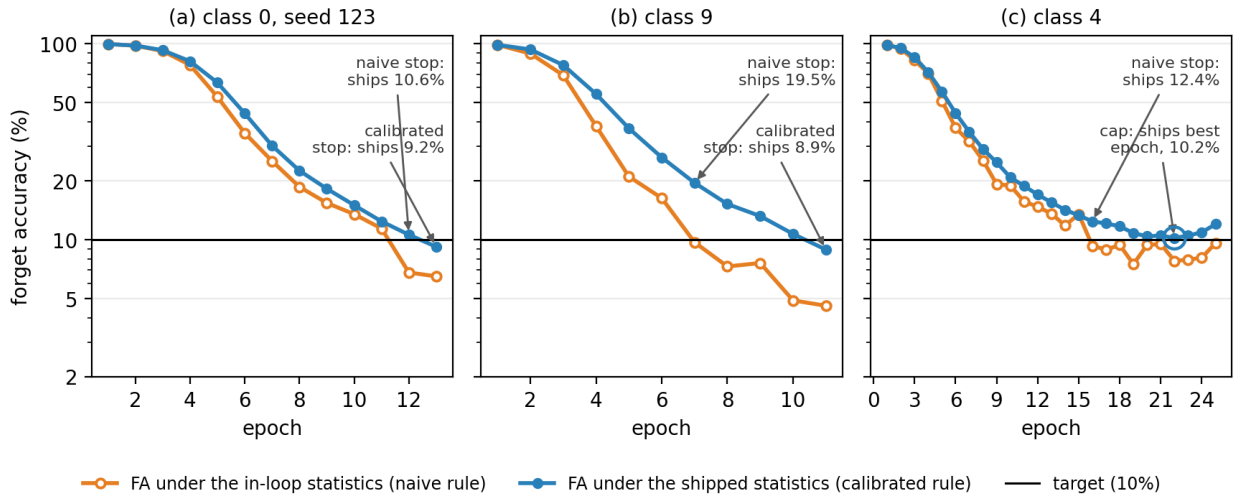


Figure 8: **Why stopping must be calibrated.** Forget accuracy per epoch under the in-loop statistics (what naive stopping reads) and under the statistics that ship (what calibrated stopping reads); log scale. The in-loop reading runs below the shipped one, so a naive rule stops early and ships above chance: (a) one epoch early on class 0, (b) four epochs early on class 9, shipping 19.5% instead of 8.9%. (c) On class 4 the shipped FA plateaus just above 10% and the run reaches the epoch cap; the best-epoch rule ships epoch 22 (10.2%) rather than the last (12.0%).

Table 10: **BN-robust PURGE under BatchNorm recalibration** (CIFAR-10, class 0, seeds $\{42, 123, 456\}$; `update_bn` on the retain set, as in Table 9). Every row shares the 93.97% base except GroupNorm, which needs its own. The un-unlearned base is the control: recalibrating it changes nothing. *Naive* stopping reads FA under the in-loop statistics, *calibrated* stopping under the statistics that ship. Time is wall-clock per run on one GPU.

Configuration	base TA	pre-F	post-F	$\boldsymbol{\Delta F}$	TA	RA	epochs	min
Base model (no unlearning)	93.97	99.80	99.82	+0.0	93.97	—	—	—
PURGE (BatchNorm, frozen)	93.97	8.51 ± 0.30	99.12 ± 0.09	**+90.6**	85.20	97.34	2	1.9
PURGE (GroupNorm)	91.89	5.91 ± 0.09	n/a	n/a	83.13	98.92	3	2.5
BN-robust, naive stopping	93.97	11.43 ± 1.90	11.44 ± 1.87	**0.0**	84.42	95.46	11–13	9.8
BN-robust, calibrated stopping	93.97	9.19 ± 0.08	9.18 ± 0.07	**0.0**	84.30	95.59	13	13.3

than 200 epochs (91.89%), and a retrained MNIST base (99.59%). Their absolute numbers are therefore not comparable row for row with the rest of §5; every comparison below is between arms that share a base.

It survives the attack it was built against, and two it was not. Calibrated BN-robust forgets class 0 to 9.19% and stays at 9.18% under recalibration (Figure 9a), while the frozen configuration returns to 99.12% with test accuracy back at 94.1%—the original model. The exponential-average operator, which is not idempotent and therefore not a no-op here, moves BN-robust by at most +0.3pp. Nor does robustness depend on the attacker using the retain set. On the naive-stopping checkpoints, recomputing the statistics on the full training set, forget class included, or on the test set—data BN-robust never trained under—moves its FA by −1.0 to −1.2pp, towards more forgetting, while the frozen configuration still recovers to 97.4% and 97.3% (Figure 9b).

Every class, and a second dataset. On all ten CIFAR-10 classes (seed 42; Table 20 and Figure 13), the frozen configuration recovers to 94.7–99.3% under the attack, while BN-robust's FA moves by at most 0.06pp under either stopping rule. With calibrated stopping every class ships between 6.7% and 10.2%, inside three standard errors of chance (11.3% for 5,000 images); with naive stopping, five of the ten did

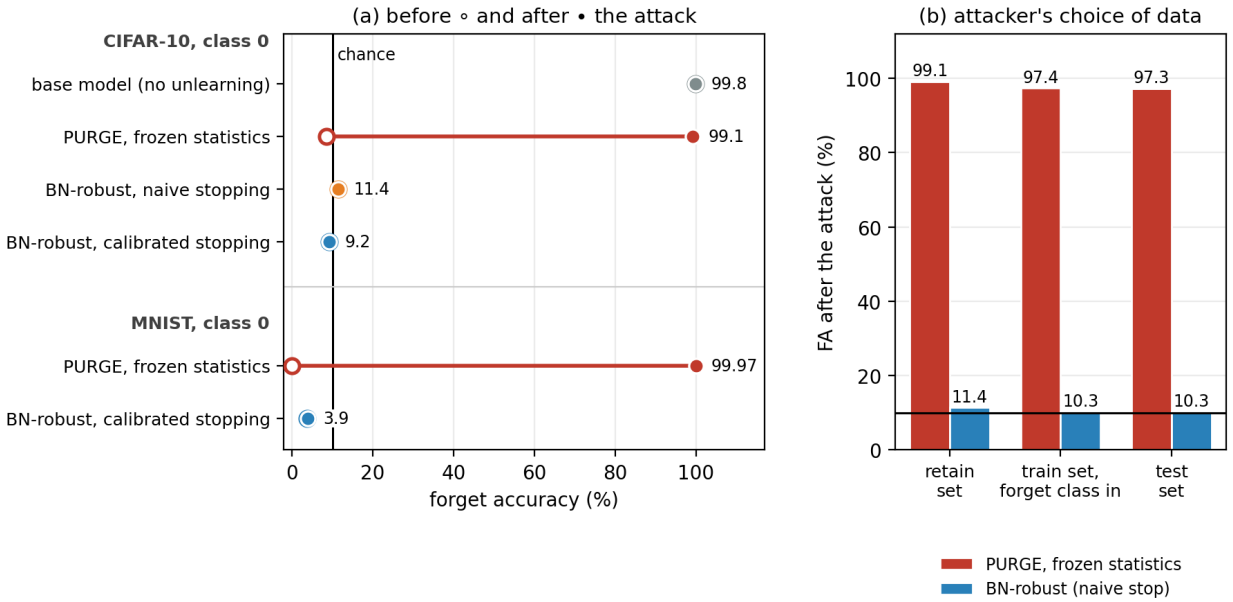


Figure 9: **BN-robust survives the recalibration attack.** (a) Forget accuracy before (open) and after (filled) `update_bn` on the retain set, class 0, means over three seeds; on MNIST the frozen configuration's forgetting is undone completely. (b) Forget accuracy after the attack when the attacker recalibrates on the retain set, the full training set (forget class included), or the test set (CIFAR-10 class 0, three seeds; BN-robust with naive stopping).

not. One class needed the fallback. Class 4 hovered at 10.2–10.9% for six epochs without crossing 10% and reached the epoch cap, and the rule shipped its best epoch (10.2%) rather than the last (11.9%, with retain accuracy 1.4pp lower). On MNIST, over three seeds, the frozen configuration's forgetting is undone completely (0.00% → 99.97%, with test accuracy back to 99.56% against the base's 99.59%), and BN-robust holds (3.78% → 3.87%, test accuracy 90.10%).

What it costs, and why it is slower. Against the frozen configuration, BN-robust gives up 1.8pp of retain accuracy and 0.9pp of test accuracy and takes about seven times the wall-clock, most of it because it needs 13 epochs instead of 2. We read the slower forgetting as the substance rather than the overhead. The frozen configuration reaches its target in two epochs largely through the statistics mismatch, which the attack shows is not forgetting at all; once the statistics track the weights, the forgetting has to happen in the weights. BN-robust's test accuracy, 84.30%, is close to the 84.6% expected from removing one class of ten from a 93.97% model. Against GroupNorm, BN-robust keeps 1.2pp more test accuracy, mostly because it can start from the stronger BatchNorm base, and it needs neither a change of architecture nor a retrained base model. Membership-inference AUROC stays between 0.48 and 0.52 for every unlearned row.

What it does and does not establish. Unlike GroupNorm or a transformer, this is robustness rather than immunity by construction: the attack is defined on the model, runs, and finds nothing to recover. It is robustness to one attack, though. BN-robust changes only how the normalisation statistics are maintained, so it does nothing against attacks that modify the weights, such as fine-tuning recovery (§7), and we have tested it on two datasets rather than six. Within that scope, PURGE no longer has to choose between BatchNorm and its forgetting: BN-robust PURGE, PURGE with GroupNorm, and PURGE on a transformer are all configurations we would deploy.

Table 11: **ViT-B/16 on CIFAR-10**, class 0, 3 seeds, same algorithm and tolerances as Table 3. The projection fires on 99–100% of steps, as it does on ResNet-18.

Backbone	TA	FA↓	RA	MIA
ResNet-18 (Table 3)	86.13 ± 0.02	1.87 ± 0.25	100.00 ± 0.00	0.529
ViT-B/16	87.05 ± 0.07	9.71 ± 0.22	97.55 ± 0.08	0.453

5.7 A Transformer Backbone

Every result so far uses ResNet-18, so the projection has only ever been applied to a convolutional network. We repeat CIFAR-10 class 0 on ViT-B/16 under the identical algorithm, with representation erasure disabled as everywhere else, over three seeds.

Forgetting transfers: FA reaches 9.71%, below the 10% chance level, with retain accuracy at 97.55% and test accuracy slightly *above* the ResNet-18 result. The gap in FA against ResNet-18's 1.87% is not a weaker result but a different stopping point—both runs use $\epsilon_F = 10\%$, and the ViT run halts as soon as it crosses that threshold (epoch 3, mid-epoch), whereas the ResNet-18 run overshoots it. §5 shows that overshoot is an artifact of check granularity rather than a property of either backbone.

Two things follow. First, the conflict the projection resolves is not an artifact of convolutional architectures: the projection fires on 99–100% of steps here as well, so forget and retain gradients oppose one another just as routinely in a transformer. Second, and more usefully, ViT normalises with LayerNorm, which computes statistics per example at inference and stores none. It is therefore outside the reach of the recalibration operator of §5.5 for the same structural reason as GroupNorm, but by way of a completely different architecture—which is what makes the pair informative rather than merely repetitive.

5.8 Component Ablation

A component ablation run in the deployed configuration is uninformative here, and the reason is instructive. Under the projected-momentum optimiser every variant reaches the forget-accuracy target inside the first check interval, so dual stopping halts them all after one epoch and each looks equally good: in the upper half of Table 12, removing the projection costs 0.09pp of retain accuracy. That number is an artefact of stopping, not a measurement of what the projection does.

We therefore also run every variant at *equal compute*, with stopping disabled throughout (lower half). The picture inverts.

The projection is what makes unlearning survivable. At equal compute, removing it costs 41.3pp of test accuracy and 47.4pp of retain accuracy: the model collapses to 44.4/52.6 against 85.7/99.98 for the full method. This is the empirical counterpart of Theorem 1, and it is a far larger effect than the stopping-confounded comparison suggests. Note also that the two rows disagree about the *sign* of the conclusion—one reports the projection as costless, the other as indispensable—which is why we report both rather than whichever is more favourable.

KD contributes little once compute is equalised. Removing the distillation anchor changes retain accuracy by 0.04pp at equal compute. We previously attributed several points of retain accuracy to it; that is not supported here, and we no longer claim it.

The removed components stay removed. Re-introducing representation erasure reproduces the full method to within 0.04pp on every metric, and re-introducing the entropy gate leaves forget accuracy at 19.3%—above the 10% chance level, i.e. it prevents the method from forgetting. Both were dropped in §3 on the strength of an ablation run under the previous implementation; this re-run confirms the decision under the current one.

Table 12: **Component ablation** (CIFAR-10 class 0, seed 42). Upper: the deployed configuration, where dual stopping halts every variant after one epoch. Lower: equal compute, 15 epochs, stopping disabled for all variants—the same comparison without the stopping confound. `+rep` and `+gate` re-introduce the two components removed in §3.

	Variant	TA	FA	RA	MIA
stopping on	full PURGE	86.10	2.14	100.00	0.528
	−KD	85.56	0.80	99.99	0.516
	−projection	84.87	0.78	99.91	0.513
	selective-layer	86.26	3.26	100.00	0.528
	+rep	86.10	2.14	100.00	0.528
	+gate	86.10	2.14	100.00	0.528
	−stopping	85.71	0.00	99.98	0.472
equal compute	full PURGE	85.70	0.00	99.98	0.472
	−KD	85.66	0.00	99.94	0.475
	−projection	**44.37**	0.00	**52.55**	0.436
	selective-layer	85.76	0.00	99.99	0.469
	+rep	85.74	0.00	99.98	0.473
	+gate	87.16	19.28	100.00	0.539

Table 13: **Projection strength** (CIFAR-10 class 0, equal compute, stopping disabled). $s = 0$ applies no correction, $s = 1$ is the full hard constraint. Forget accuracy is 0.0% throughout, so the runs differ only in collateral damage.

s	residual conflict	TA	FA	RA	MIA
0.00	$1.00\,\langle v_t, g_\mathrm{r}\rangle$	44.35	0.0	52.54	0.436
0.25	$0.75\,\langle v_t, g_\mathrm{r}\rangle$	67.84	0.0	81.35	0.497
0.50	$0.50\,\langle v_t, g_\mathrm{r}\rangle$	78.03	0.0	92.75	0.510
0.75	$0.25\,\langle v_t, g_\mathrm{r}\rangle$	82.99	0.0	98.24	0.509
1.00	**0**	**85.71**	0.0	**99.98**	0.472

5.9 How Hard Must the Constraint Be?

Theorem 2(iv) argues that a soft conflict penalty biases the update without excluding retain-harming steps, and that only a hard constraint makes them unreachable. That is an argument about feasible sets; here we measure what it costs in practice. We interpolate the projection with a strength parameter s: at each conflicting step the update becomes $u_t = v_t - s\,\frac{\langle v_t, g_\mathrm{r}\rangle}{\|g_\mathrm{r}\|^2} g_\mathrm{r}$, so

$$\langle u_t, g_\mathrm{r}\rangle \;=\; (1 - s)\,\langle v_t, g_\mathrm{r}\rangle. \tag{7}$$

At $s = 0$ nothing is corrected; at $s = 1$ this is PURGE. Every intermediate value is a soft penalty—it shrinks the conflicting component without removing it—and by Eq. 7 it leaves $\langle u_t, g_\mathrm{r}\rangle < 0$ whenever the raw update conflicts. All other settings are held fixed, and stopping is disabled so that every variant sees the same compute.

Softening is expensive, and monotonically so. Retain accuracy falls from 99.98% at $s = 1$ to 52.54% at $s = 0$, and test accuracy from 85.71% to 44.35%, monotonically in between. Because forget accuracy is 0.0% at every setting, this is not a privacy–utility trade-off: all five runs forget equally, and they differ only in how much of the retained model they destroy on the way. The $s = 0$ row reproduces the equal-compute −projection ablation of Table 12 from an independent direction.

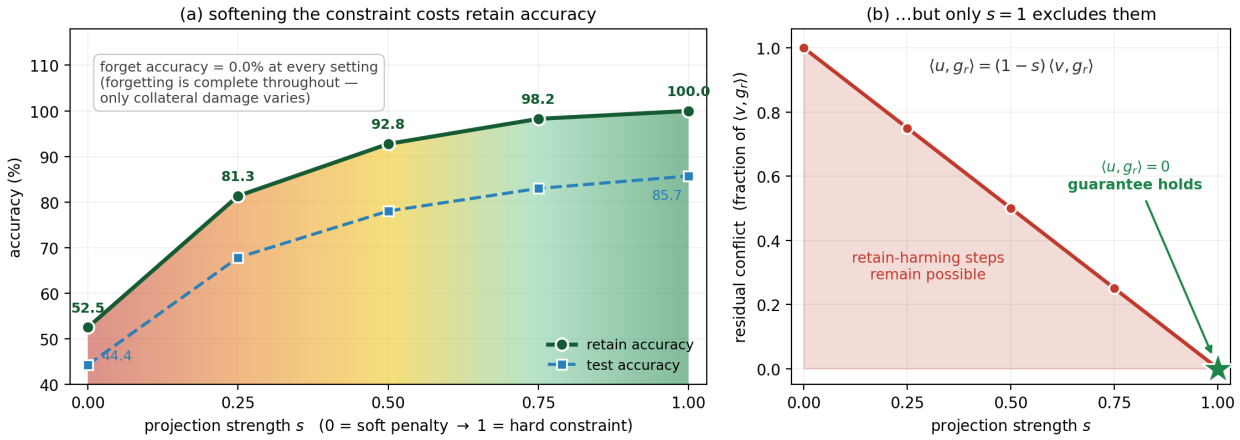


Figure 10: **Hardening the constraint recovers retain accuracy continuously, but only the endpoint carries the guarantee. (a)** Retain and test accuracy against projection strength; forget accuracy is 0.0% at every setting, so the runs differ only in collateral damage. **(b)** Residual conflict $(1 - s)$: retain-harming steps remain reachable for every $s < 1$ and are excluded only at $s = 1$. The empirical difference between $s = 0.75$ and $s = 1$ is 1.7pp of retain accuracy; the difference in what can be proved is categorical.

The curve is gradual, which we take as a limitation of the empirical argument. Most of the benefit arrives early—$s = 0.25$ already recovers 28.8pp of retain accuracy, while the final quarter, $s = 0.75 \to 1$, is worth 1.7pp. A soft penalty tuned near $s = 0.75$ therefore performs almost as well as the hard constraint on these metrics, and we do not claim otherwise. What it does not do is satisfy Eq. 7 with equality: at $s = 0.75$ a quarter of every conflict survives into the applied update, so $\langle u_t, g_r \rangle < 0$ on conflicting steps and no per-step statement can be made. The case for $s = 1$ is therefore not that the last 1.7pp is large, but that it is the only setting about which anything can be proved—which is the whole content of Theorem 1.

Relation to soft conflict-resolution methods. This is a controlled test of the mechanism that separates PURGE from the closest prior work [Patel and Qiu, 2025], not a comparison with that method: it varies our own projection rather than reimplementing theirs, and their penalty acts on the objective rather than on the update. What it establishes is the general shape of the trade—that discouraging conflict helps in proportion to how much of it is removed, and that removing all of it is what converts an empirical tendency into a guarantee. A like-for-like comparison remains future work (§7).

5.10 Privacy–Utility Frontier and Robustness Diagnostics

We sweep the two tolerances of Eq. 2 independently on CIFAR-10 class 0 (Table 14). The results are asymmetric: one tolerance is inert because the projection has already done its work, while the other is genuinely active but was hidden by how coarsely we measured it.

The retain budget is non-binding. Varying α over four orders of magnitude changes nothing: the retain-loss constraint is never the active one, because the projection already prevents the retain loss from rising. This is consistent with Theorem 1 and means α can be left at its default.

The forget target is a usable knob, but only if it is measured finely enough. Sweep B contains no frontier: every target from 5% to 30% yields the identical operating point, FA = 1.88%. We previously read this as evidence that ϵ_F is not a knob at all. That reading was wrong, and Sweep B′ shows why: the sweep was limited by the *measurement*, not by the method.

Forget accuracy is checked every 50 batches, and under this optimiser it falls from ∼99% to below 2% inside a single such interval, so every target in between is passed unobserved and the run halts at whichever

Table 14: **Tolerance sweeps** (CIFAR-10 class 0). Sweep A varies the retain-loss budget α at fixed $\epsilon_F = 10\%$; Sweeps B and B′ vary the forget-accuracy target ϵ_F at fixed $\alpha = 5$, checking forget accuracy every 50 batches and every step respectively. α is inert. So is ϵ_F under the coarse check—but only because forget accuracy crosses the whole range between two consecutive checks; measured per step, it tracks the requested tolerance across a 29pp range.

Sweep A: α	TA	FA	RA	MIA
1.5	86.10	1.88	100.00	0.530
2	86.10	1.88	100.00	0.530
3	86.10	1.88	100.00	0.530
5	86.10	1.88	100.00	0.530
10	86.10	1.88	100.00	0.530
9999	86.10	1.88	100.00	0.530
Sweep B: ϵ_F, check every 50 batches	TA	FA	RA	MIA
1%	85.75	0.28	100.00	0.521
5%	86.10	1.88	100.00	0.530
10%	86.10	1.88	100.00	0.530
20%	86.10	1.88	100.00	0.530
30%	86.10	1.88	100.00	0.530
Sweep B′: ϵ_F, check every step	TA	FA	RA	MIA
1%	85.95	**0.60**	100.00	0.531
5%	86.34	**3.86**	100.00	0.528
10%	86.59	**8.24**	100.00	0.522
20%	87.22	**16.24**	100.00	0.518
30%	88.10	**29.60**	100.00	0.515

point the next check happens to catch. Sweep B′ repeats the sweep with the check evaluated at every step—on a subsample of $\mathcal{D}_f$, with any trigger confirmed on the full forget set before stopping—and the frontier appears: forget accuracy tracks the requested tolerance monotonically from 0.60% to 29.60%, a 29pp range, with test accuracy rising from 85.95% to 88.10% as the tolerance is relaxed and retain accuracy pinned at 100.00% throughout. At $\epsilon_F = 30\%$ the coarse check lands at FA = 1.88% where 29.60% was reachable: a 27.5pp overshoot attributable entirely to measurement granularity.

Step-level traces confirm that nothing about the collapse forbids resolving it. The largest single-step change in forget accuracy is 11.7–11.9pp across the five targets—the runs follow one trajectory and differ only in where they leave it. A 12pp-per-step descent is comfortably resolvable at per-step granularity; it was invisible at 50-batch granularity. We therefore restore the ϵ_F entry in Table 1, with the qualification that the tolerance is honoured only when the stopping rule is evaluated at a resolution finer than the descent it is meant to catch. The cost is one forward pass over a 512-sample subsample per step.

MIA in isolation is uninformative. An early sign of this: the *pre-unlearning* base model already attains MIA AUROC = 0.533 at FA = 99.7%. For well-generalising classifiers, per-sample membership carries little signal at the class level, so MIA AUROC alone cannot certify class-level unlearning and must be paired with FA. §5.12 develops this into a sharper statement: under the standard protocol the attack scores *higher* against a model that never saw the data than against one that memorised it.

5.11 Distance to a Retrained Model

Forget accuracy asks whether the forget class is still predicted. It does not ask whether the unlearned model *behaves like* one trained without that class, which is what unlearning is for. We therefore retrain a CIFAR-10 reference from scratch on the retain set only, keeping all K output heads so that output distributions remain comparable, and measure how far each unlearned model sits from it (Table 15). Two spaces are measured:

Table 15: **Distance to a retrained CIFAR-10 reference** (class 0). The reference is trained on the retain set only. JS and agreement compare output distributions on forget-class inputs; Fréchet compares penultimate features. Best per column in bold. Parameter distance is reported for completeness only (see text).

Method	JS↓	agree %↑	Fréchet↓	$\|\Delta\theta\|_2$	FA	RA	$\|\text{MIA}-0.5\|$ ↓
SalUn	**0.220**	**56.5**	495.3	80.6	0.00	97.70	0.075
Bad Teacher	0.263	47.1	153.0	89.7	24.36	89.46	**0.029**
SCRUB	0.562	8.7	165.9	56.6	89.28	97.83	0.030
PURGE	0.273	35.2	**13.0**	43.2	1.76	**100.00**	0.030

the output distribution on forget-class inputs (Jensen–Shannon divergence, and top-1 agreement with the reference's predictions), and the penultimate feature distribution (Fréchet distance).

The two spaces disagree, and no method dominates. PURGE is closest to the reference in feature space by a wide margin—a Fréchet distance of 13.0 against 153–495 for the baselines, a factor of 38 against SalUn—and it is the only method that leaves retain accuracy untouched. SalUn is closest in output space, with the lowest JS divergence (0.220 against our 0.273) and the highest agreement with the reference's predictions on forget inputs (56.5% against our 35.2%). We report this plainly: on the metric that most directly operationalises "behaves like a retrained model" on forget inputs, PURGE is second.

The disagreement is itself the finding. A method can route forget-class inputs much as a retrained model would while carrying internal representations far from it, or preserve those representations while routing inputs differently. Reporting a single distance would have concealed this, and which number one reports would have determined which method appeared better. We therefore recommend that distance-to-retrained be reported in both spaces, and we do so.

SCRUB is far away because it has not unlearned. Its JS divergence is more than twice ours and its agreement with the reference is 8.7%, but this follows from FA = 89.3%: a model that still classifies the forget class correctly cannot resemble one that never saw it. This is a useful sanity check on the metric—it ranks non-unlearning last—rather than a criticism of SCRUB under a different operating point.

Parameter distance is uninformative. $\|\Delta\theta\|_2$ orders the methods differently again (43.2 for PURGE, 56.6 for SCRUB, 80.6 for SalUn, 89.7 for Bad Teacher) and tracks neither behavioural measure. Permutation and scaling symmetries mean two functionally identical networks can sit far apart in parameter space, so we report it for completeness and draw no conclusion from it.

5.12 Membership Inference, Recalibrated

Every MIA number reported so far, in this paper and in much of the unlearning literature, is read against a target of 0.5: the attack should be unable to distinguish forget-class training samples from forget-class test samples. We ran four attacks against a retrained reference to check that target, and it does not survive.

We evaluate four threshold-free signals—negative loss, maximum confidence, negative entropy, and modified entropy [Song and Mittal, 2021], the last of which uses the true label and so separates confidently-correct from confidently-wrong predictions—and report AUROC together with true-positive rate at 1% false-positive rate. The second matters because an attack that identifies a few members with near-certainty is a real privacy failure that an average AUROC of 0.5 can conceal.

The retrained reference does not score 0.5**.** It scores 0.694 on the loss attack, with a true-positive rate of 12.5% at 1% false positives. This model trained on *neither* group: to it, forget train and forget test are both non-members. The attack is therefore not detecting membership. It is detecting that the CIFAR-10 train and test splits of class 0 differ in difficulty, and that difference is visible through any model that has

Table 16: **MIA suite on CIFAR-10 (class 0).** Members are forget-class training samples, non-members forget-class test samples (5000 vs 1000). The retrained reference trained on *neither* group, so it is the empirical target; the base model trained on the members and is the opposite anchor. TPR is at 1% FPR.

		loss		mentr		conf		entropy
Model	FA	AUROC	TPR	AUROC	TPR	AUROC	TPR	AUROC
retrained (reference)	0.0	0.694	12.5%	0.646	9.9%	0.435	0.1%	0.422
base (no unlearning)	100.0	0.483	0.0%	0.486	0.0%	0.482	0.0%	0.479
SalUn	0.0	0.500	0.0%	0.537	2.3%	0.463	0.0%	0.461
Bad Teacher	24.8	0.485	0.2%	0.493	0.2%	0.455	0.6%	0.444
SCRUB	89.5	0.477	0.4%	0.478	0.4%	0.467	0.4%	0.462
PURGE	2.0	0.693	1.4%	0.691	1.3%	0.369	0.0%	0.340

not memorised either. A target of 0.5 is wrong for this protocol, and 0.694 is the level a perfectly unlearned model actually exhibits.

The attack works *better* against the model that never saw the data. The base model memorised all 5000 members and classifies them at 100%; the standard intuition says it should be the most attackable thing in the table. It scores 0.483—below chance, and 0.21 below the retrained reference. A well-generalising classifier treats members and non-members of a class alike, leaving no signal, whereas a model that never saw the class makes errors whose pattern differs between the splits. Under this protocol, MIA AUROC is anti-correlated with memorisation.

The choice of target reverses the ranking. Read against 0.5, PURGE looks like the *worst* method in the table ($|\text{AUROC} - 0.5| = 0.193$) and SalUn like the best (0.000). Read against the retrained reference—the model unlearning is supposed to imitate—the order inverts: PURGE is nearest ($|\text{AUROC} - \text{ref}| = 0.001$) and SalUn is 0.194 away. We report both columns rather than the flattering one, and we note that our own earlier claim of "MIA within 0.035 of the ideal 0.5" was resting on the wrong reference point.

We do not want to over-read the agreement between PURGE and the reference. It is one dataset, one forgotten class, one protocol, and matching to 0.001 is closer than the measurement warrants; what the data supports is that PURGE sits near the retrained model on this attack while the baselines sit near 0.5, not that it matches it exactly.

On a second dataset the reference is not even a *constant*. The argument above rests on one dataset and one class, which is thin for a claim about everyone's protocol, so we repeated it on PathMNIST, retraining a reference for each of four classes (Table 17). The finding is sharper than we expected. The reference AUROC is not merely different from 0.5; it moves with the forgotten class, and by a large amount. On the loss attack it ranges from 0.306 (class 2) to 0.704 (class 0), a span of 0.398—wider than the entire distance from 0.5 to either endpoint. The other three attacks span 0.31–0.36 similarly, and reverse direction: on class 0 the reference sits *above* 0.5 for loss and *below* it for confidence, while on classes 2–4 that ordering flips.

This strengthens the diagnosis considerably. It is not that the field uses the wrong constant; it is that no constant is correct. Any protocol that scores unlearning by $|\text{AUROC} - 0.5|$ is comparing against a quantity that varies by up to 0.4 with the choice of forgotten class, so the same method can be judged excellent or poor on the same dataset depending on which class the deletion request happens to name.

Our own matching result does not generalise, and we withdraw the general form of it. The CIFAR-10 agreement above (0.001 on the loss attack) is not reproduced on PathMNIST. PURGE matches its class-0 reference closely (mean gap 0.040, comparable to CIFAR-10's 0.049), but on classes 2, 3 and 4 the mean gaps are 0.237, 0.359 and 0.218—an average of 0.213 across the four classes, four times the CIFAR-10

Table 17: **MIA on PathMNIST, four classes, each against its own retrained reference.** The reference is the level a perfectly unlearned model exhibits. There is no single such level: it depends on which class was forgotten, and the attacks disagree on direction. Gap is the mean over the four attacks of $|\text{AUROC}_{\text{PURGE}} - \text{AUROC}_{\text{ref}}|$.

Class	Model	loss	mentr	conf	entropy	gap
0	retrained (reference)	0.704	0.684	0.345	0.344	—
	PURGE	0.711	0.719	0.305	0.267	**0.040**
2	retrained (reference)	0.306	0.360	0.563	0.571	—
	PURGE	0.403	0.407	0.187	0.141	0.237
3	retrained (reference)	0.345	0.329	0.666	0.657	—
	PURGE	0.569	0.597	0.201	0.180	0.359
4	retrained (reference)	0.359	0.366	0.614	0.617	—
	PURGE	0.589	0.598	0.412	0.410	0.218
reference range across classes		0.398	0.355	0.321	0.312	

figure. PURGE's own AUROC is comparatively stable across classes while the reference is not, so the method does not track a moving target.

We therefore restrict the claim: on CIFAR-10, and on the class of PathMNIST where the reference resembles CIFAR-10's, PURGE sits close to the retrained model on loss-based attacks; we do not claim this holds across forgotten classes in general, and the PathMNIST measurements show that it does not. The protocol finding and the method finding should be read separately—the first is strengthened by this experiment, and the second is narrowed by it.

An AUROC of exactly 0.5 can be degenerate. SalUn's loss attack gives 0.5000 with a true-positive rate of 0.0% at every false-positive rate we measured. An uninformative-but-varying score gives TPR $\approx$ FPR, i.e. about 1%; a TPR of zero is the signature of a *constant* score. SalUn maps every forget-class input to the same output, so there is nothing for the attack to rank. This is a way of being unattackable, but it is not evidence of having imitated a retrained model—and reporting AUROC alone cannot tell the two apart. The TPR column can.

Recommendation. For class-level unlearning we recommend that MIA be reported (i) with more than one attack, since the four disagree here even in sign—the reference is above 0.5 on loss and below it on confidence, and that ordering itself flips between PathMNIST classes; (ii) with TPR at a low FPR alongside AUROC, since an AUROC of 0.5 can be produced by a constant score that no attack can rank; (iii) calibrated against a retrained reference rather than against 0.5; and (iv) *per forgotten class*, against that class's own reference, because Table 17 shows a single reference does not transfer between classes of one dataset.

Point (iv) is expensive—it costs one retraining per class—which is the honest obstacle to the recommendation. Where that is out of reach, we would report MIA as a diagnostic alongside forget accuracy and decline to draw any conclusion from its distance to 0.5, rather than treat 0.5 as a target the measurements do not support.

5.13 Over-Forgetting, and a Fix That Did Not Work

Two PathMNIST classes are over-forgotten: MIA AUROC reaches 0.298 and 0.388, i.e. forget-class training samples become *harder* to classify than forget-class test samples. Such a model is still distinguishable from a retrained one—in the opposite direction to a privacy leak—while satisfying every forget-accuracy check. FA cannot detect this, because moving *to* the distribution a retrained model produces and moving *past* it both give FA ≈ 0.

We hypothesised that the cause was the gap between our forget-side constraint as *stated* in Eq. 2, $\text{KL}(p_\theta(x_f)\|p_r) \leq \epsilon_F$, and as *implemented* by dual stopping, FA $\leq \epsilon_F$, and that stopping on the divergence

instead would fix it. We retrained reference models without the forget class, pre-committed to the prediction, and ran the experiment. **The prediction failed.** Stopping on the divergence was worse on 10 of 12 comparisons, made Jensen–Shannon divergence to the retrained reference worse on all three classes, and degraded an unaffected control class—the signature that rules out a targeted fix. Across checkpoints the proposed criterion is rank-*anti*-correlated with distance to the retrained model ($\rho = -1.00$): driving it toward zero moves the model away from the reference. Appendix F reports the experiment in full, including a limitation of the test itself and a confound we cannot separate.

Two conclusions follow, and we adopt both. First, a narrower reading of the `kl_retain` objective: p_r is a useful forget-side *direction* (§3.3), but it is not what a retrained model produces on forget inputs, and minimising the divergence to it is actively harmful. The constraint in Eq. 2 should therefore be read as a tolerance satisfied early, not as an objective to be driven to its minimum. Second, the experiment localised a different and more mundane cause: all three FA-stopped runs overshoot their own 11.1% target by ∼4pp because the intra-epoch FA check is evaluated only every 50 batches, and divergence to the retrained model is lower at the earlier stopping point in every class. The indicated fix is to stop earlier and more precisely rather than to replace the stopped quantity.

That fix is now implemented and it works. Evaluating the check at every step rather than every 50 batches removes the overshoot on CIFAR-10 and recovers a monotonic frontier over a 29pp range of forget accuracy (§5, Sweep B′ of Table 14); the descent it must resolve is ∼12pp per step, which is well within reach at that granularity. The mechanism is shared, so we expect the same remedy to reduce the PathMNIST overshoot, but we have not re-run the over-forgotten classes under the finer check and do not claim the over-forgetting defect itself is resolved. What has changed is its status: it is a consequence of measuring the stopping criterion too coarsely, not evidence that the criterion is the wrong one. The remaining question—where along the trajectory divergence to the retrained model is minimised, and whether a single FA level identifies it across classes—stays open.

5.14 Locating the Stopping Point, and What MIA Cannot Tell Us

The refuted hypothesis of §5.13 left a concrete question: if stopping on the forget-side divergence is not the fix, *where* should unlearning stop? We answer it by measuring the whole trajectory rather than comparing two rules. For PathMNIST classes 2, 3 (over-forgotten) and 4 (control), we run unlearning with stopping disabled, snapshot the weights every 8 optimiser steps, and evaluate each snapshot against a retrained reference (Appendix F). Figure 11 shows the result.

The minimum is interior, and the current rule is past it. Divergence to the reference falls from 0.66–0.69 at initialisation to 0.28–0.38, then rises again to 0.40–0.42. So unlearning genuinely moves the model toward a retrained one before moving away: over-forgetting is not a metric artefact but a real overshoot in function space. The minimum lies at FA = 11.8%, 14.3% and 12.6% for the three classes, while dual stopping halts at ≈7% (§5.13) — past the optimum in every case. Stopping at the minimum instead is close to free: test accuracy changes by −0.06, −0.03 and +0.79pp and retain accuracy by +0.06, +0.36 and +0.08pp. On the same trajectory it reduces divergence by 0.016, 0.003 and 0.007, and reduces $|\text{MIA} - 0.5|$ by 0.017 and 0.015 on the two affected classes while moving the control by +0.006. This is the pattern the criterion of §5.13 failed to produce: the affected classes improve and the control stays put.

But the optimum is shallow, so we do not propose a numeric target. The three minima fall within 2.5pp of one another in forget accuracy, which invites the conclusion that "stop at FA ≈ 13%" is a deployable rule. The basin does not support that reading. Taking every checkpoint within 5% of the minimum divergence, the corresponding forget accuracy spans 10.2–26.3%, 3.5–36.7% and 6.9–21.9% on the three classes—3.5–36.7% overall. The agreement between the three *argmin* values is therefore largely a coincidence of where the snapshots happened to land, and a single tuned number would not transfer. What the data does support is directional and, we think, more useful: the optimum sits *above* the chance level, and a rule that drives forget accuracy to or below chance is on the wrong side of it.

$|\text{MIA} - 0.5|$ is not an optimisation target. Panel (b) is the more consequential finding. MIA AUROC does not converge to 0.5; it *passes through* it, falling from 0.82, 0.74 and 0.63 at initialisation to 0.22, 0.39

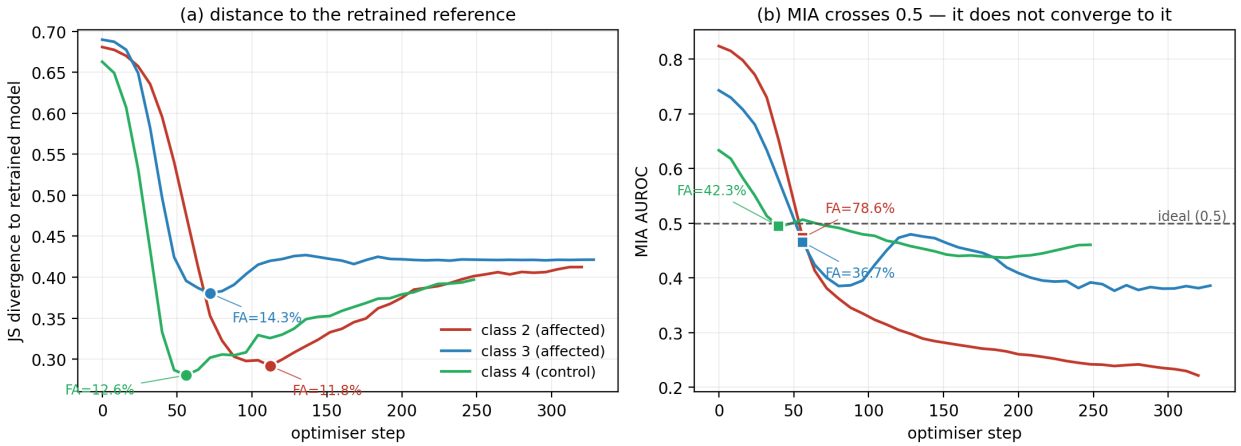


Figure 11: **Unlearning overshoots the model it is trying to imitate, and MIA cannot detect it. (a)** Jensen–Shannon divergence to a retrained-from-scratch reference along the unlearning trajectory. The curve is U-shaped: the model first moves toward the reference, attains a minimum, then moves away. Markers give the minimum, annotated with the forget accuracy there (11.8–14.3%)—*above* the 11.1% chance level, whereas dual stopping halts at ≈7%. **(b)** MIA AUROC on the same axis. It sweeps *through* the ideal 0.5 rather than converging to it, so $|\text{MIA} - 0.5|$ is minimised at the crossing. On class 2 that crossing occurs while the model still classifies 78.6% of the forget set correctly.

and 0.46 at the end. Consequently $|\text{MIA} - 0.5|$ is minimised wherever the trajectory happens to cross, which carries no information about unlearning quality. On class 2 the crossing occurs at FA = 78.6%: a model that has barely forgotten anything attains a *perfect* MIA score. The two criteria also disagree about where to stop, and in opposite directions—the divergence minimum and the MIA minimum are 56 steps apart on class 2, 56 steps apart the other way on class 3, and 8 steps apart on class 4.

This sharpens the observation of §5 that a pre-unlearning model already scores MIA = 0.533. That was a single point; here we see the whole path, and it shows that MIA ≈ 0.5 is attainable at almost any stage of unlearning, including stages with no meaningful forgetting at all. Reporting MIA alongside forget accuracy is therefore not merely advisable but necessary, and *optimising* $|\text{MIA} - 0.5|$—which a practitioner reading "ideal = 0.5" might reasonably attempt—selects an arbitrary point on the trajectory.

What remains unresolved. Stopping at the divergence minimum reduces over-forgetting without eliminating it: class 2 still attains MIA = 0.315 there, far from 0.5. Nor do we have a deployable rule, since locating the minimum requires the retrained reference that unlearning exists to avoid, and the basin is too flat for a fixed forget accuracy to stand in for it. We report the diagnosis—the stopping rule halts on the wrong side of a real optimum—and leave a practical criterion open. Whether the effect holds beyond the three PathMNIST classes measured here is also untested; the experiment costs one unlearning run plus one retrained reference per class.

6 Related Work

Exact unlearning. SISA [Bourtoule et al., 2021] partitions data into shards so that removal requires retraining only the affected shard; the per-deletion cost approaches full retraining as shard count grows. Certified removal [Guo et al., 2020] gives exact guarantees for convex models but does not extend to deep networks.

Gradient-based approximate unlearning. GA [Thudi et al., 2022] maximises forget-set loss, with no mechanism to prevent the same direction from degrading retain performance. Amnesiac unlearning [Graves et al., 2021] subtracts cached per-sample updates, requiring $O(T|\mathcal{D}_f|)$ storage. NegGrad+ optimises

$-\mathcal{L}_\mathrm{f} + \lambda\mathcal{L}_\mathrm{r}$ directly, which is the Lagrangian relaxation of Eq. 2 rather than a solution to it: the retain term shifts the optimum without bounding any single step, so the weighted sum is exactly the soft treatment Theorem 2(iv) separates us from.

Saliency and Fisher methods. SalUn [Fan et al., 2024] applies random-label updates inside a top-50% saliency mask: strong empirically (FA = 0% on CIFAR-10) but with no formal retain guarantee and no control over the degree of forgetting. Fisher-based methods [Golatkar et al., 2020, 2021] and SSD [Foster et al., 2024] weight parameters by estimated importance, requiring Hessian approximations that scale poorly.

Knowledge distillation. Bad Teacher [Chundawat et al., 2023] uses competent/incompetent teachers. SCRUB [Kurmanji et al., 2023] is closest in motivation—both draw on CL intuitions and use KD to anchor retain behaviour—but differs algorithmically: SCRUB alternates forget and retain steps balanced by λ, so its balance is hyperparameter-dependent, whereas PURGE's retain safety holds by construction (Theorem 2); and SCRUB ascends the forget loss, which is unstable in either direction, whereas `kl_retain` converges to a specific target distribution.

Continual learning connection, and the closest prior work. A-GEM [Chaudhry et al., 2019] projects task gradients to avoid catastrophic forgetting. The CL–MU duality has been noted conceptually [Nguyen et al., 2020, Kurmanji et al., 2023]. Closest to PURGE, recent work [Patel and Qiu, 2025] also targets the forget–retain gradient conflict, but through an *implicit, soft* regulariser encouraging a positive inner product—with no per-step constraint and no formal guarantee. More broadly, gradient projection and conflict resolution are now actively explored for unlearning, but as heuristics rather than as constrained updates carrying a guarantee. PURGE enforces $\langle u_t, g_\mathrm{r}\rangle \geq 0$ explicitly at every step, on the post-momentum update, which is the difference between *softly discouraging* conflict and *provably eliminating* it—and, relative to guarantees stated for momentum-free surrogates, the difference between a bound that describes the deployed optimiser and one that does not. §5.9 tests the mechanism this distinction rests on by varying our own projection between a soft penalty and the hard constraint; a like-for-like run of that method on our stack remains future work.

7 Limitations

Compute scales with classes and with data scarcity, and we cannot predict it. At the shared 15-epoch budget PURGE leaves STL-10 at FA = 75.7%, and an earlier version of this paper reported that as a failure mode of the hard constraint on small forget sets. That diagnosis was wrong in its conclusion though right about its cause. With 500 training images per class the forget set yields only 4 mini-batches per epoch, ∼60 update steps in total, and every step conflicts with the retain gradient—so progress per step is small and the budget expires before forgetting completes. At 60 epochs, over the same three seeds, FA falls to $1.73 \pm 0.77\%$ with RA = 98.87%. CIFAR-100 behaves the same way for a different reason: its per-class forget sets are small because there are a hundred of them, and at 60 epochs its nine-class mean falls from 4.07% to $1.27 \pm 0.37\%$, bringing the 1% chance level inside one standard deviation. Neither constraint was infeasible; both runs were truncated.

What survives as a limitation is that we cannot say in advance how much budget a new dataset needs. The binding resource is optimiser steps rather than epochs or examples, and PURGE offers no way to predict the requirement beyond monitoring forget accuracy and continuing until the tolerance is met. A practitioner deploying on an unfamiliar dataset should expect to discover the budget empirically, and should treat a shared epoch count copied from another dataset as a likely under-estimate.

Sample-level forgetting works, but at a cost the method cannot control. Every result in this paper removes a whole class. We tested the other standard request type—forgetting a random subset of training points—on CIFAR-10 at 1% and 5% of the training set, and report it as a scope limit rather than a capability.

Two things go wrong, and they are different in kind. First, the forget objective degenerates. `kl_retain` draws its power from the forget class being *absent* from the retain set, which drives p_r's mass on that class

to ∼0 (measured: 0.0002); when the forget set is a random subset, the retain set still covers every class and p_r is uniform to three decimals $(0.100, 0.100, \dots)$. `kl_retain` therefore reduces exactly to `kl_uniform` in this regime, and the target carries no information about *which* points to forget. Second, dual stopping does not fire: the retain budget is multiplicative on an initial retain loss of 0.0018, so even at $\alpha = 100$ both stopping-armed runs halted on the budget with forget accuracy still at ∼99.5%. A multiplicative budget is the wrong instrument against a converged model; an absolute floor would be the right one.

Given a fixed 60-epoch budget and no stopping, forgetting does occur—forget accuracy falls to 58.4% at 1% and 71.8% at 5%—but indiscriminately: retain accuracy falls from ∼100% to 73.2% and test accuracy to 71.9%, and forget accuracy ends *below* test accuracy, which is over-forgetting by the standard for this setting (a correctly unlearned point should look like an unseen point of the same class, i.e. FA ≈ TA). The method has no mechanism to stop at that crossing.

The one clearly positive finding is the ablation. Removing the projection at equal budget collapses the model completely: retain accuracy 14.2% versus 73.2% with it, and test accuracy 17.1% versus 71.9%—a 59pp and 55pp difference respectively, far larger than the projection's effect in the class-level ablation of Table 12. When forget and retain gradients are drawn from the same distribution the conflict is near-total (the projection fires on 100% of steps), and the retain constraint is the only thing keeping the model intact. The guarantee is therefore *more* load-bearing here, not less—which is why we regard sample-level forgetting as a direction the constraint supports and the objective does not yet.

Mini-batch scope of the guarantee. Theorem 1 bounds the *sampled* retain objective—the quantity PURGE projects against—not the full-population retain loss. A population-exact per-step guarantee would require controlling the gradient-variance term and is future work.

The comparison with the closest prior work is structural, not empirical. Our separation from soft conflict-resolution methods [Patel and Qiu, 2025] rests on Theorem 2(iv) and Table 1, both of which argue from the definition of the update rule. We have not reimplemented that method and run it on our stack, so we make no claim about how the two compare on accuracy, forgetting, or MIA. A like-for-like empirical comparison is the most important piece of missing evidence in this paper. Two related experiments are reported and should not be mistaken for it: the projection-only ablation, isolating the projection from PURGE's other three components (Table 4), and the projection-strength sweep, which interpolates our own update between a soft penalty and the hard constraint (§5.9). Both test the mechanism; neither runs the cited method.

Floating-point exactness. The guarantee holds to float32 precision (§5.4); exact arithmetic would require float64 inner products.

BatchNorm robustness is bought with compute, and is scoped. BN-robust PURGE (§5.6) survives the recalibration attack at about seven times the wall-clock of the default configuration, because forgetting that has to happen in the weights takes more steps. It was evaluated on two datasets, CIFAR-10 (every class) and MNIST, and on base models that differ from those behind the rest of the paper. It also defends only the normalisation statistics: an attacker who modifies weights, as in the fine-tuning recovery below, is outside what it changes.

Utility gap and shared vulnerabilities. A test-accuracy gap versus SalUn reflects the cost of conservative, projection-constrained forgetting. Two vulnerabilities shared with other weight-modification methods persist under the corrected implementation. *Sequential unlearning* degrades cumulatively: forgetting classes 0, then 1, then 2 from each resulting checkpoint drives test accuracy 86.1 → 76.8 → 69.1 and retain accuracy 100.0 → 89.7 → 80.7, while forget accuracy stays low at every step (≤ 2.7%). *Fine-tuning recovery*: retraining the unlearned model on 50 forget-class samples raises forget accuracy from 1.9% to 7.1% after five epochs and 51.3% after ten—though by then the model is no longer usable (TA = 34.6%, RA = 47.9%), so the attack recovers the class only by destroying the model it attacks. PURGE also does not provide formal (ε, δ)-DP guarantees.

8 Conclusion

We presented PURGE, built on the observation that unlearning is the dual of continual learning, and argued that for a per-step guarantee to be meaningful it must be stated for the update the optimiser actually applies. Projecting the post-momentum update rather than the raw gradient achieves this: the guarantee of Theorem 1 is exact for the deployed optimiser, and the momentum ablation shows the alternative—removing momentum to simplify the analysis—breaks forgetting on five of six datasets. Across six datasets and 31 forgetting tasks, PURGE attains per-dataset mean FA $\leq 6.2\%$ at a shared epoch budget—STL-10 needs four times that budget, at which it reaches 2.8%—with RA $\geq 97.2\%$ (92.2% on CIFAR-100), while step-level logging confirms the constraint is active at every step and satisfied at every step. On CIFAR-10, calibrated against a retrained reference rather than against 0.5, PURGE is the closest of the methods compared on the loss-based membership-inference attack; on PathMNIST, where the reference itself varies by 0.4 across forgotten classes, that agreement holds for one class of four and we do not claim it in general. On BatchNorm backbones, the forgetting PURGE achieves against frozen statistics does not survive recomputing them. Recomputing them throughout unlearning instead, and stopping on the forget accuracy the shipped model will actually have, makes it survive on every CIFAR-10 class and on MNIST.

More broadly, the CL–MU duality is an underexploited design space: any CL technique that protects old-task knowledge has a natural MU counterpart that protects retained knowledge during erasure. We also draw a methodological conclusion that generalises beyond this method—when an optimiser detail (momentum, clipping order, projection magnitude) is chosen for analytical convenience, it should be checked against the behaviour it was supposed to explain.

References

Lucas Bourtoule, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.

Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations (ICLR)*, 2019.

Vikram S Chundawat, Ayush K Tarun, Murari Mandal, and Mohan Kankanhalli. Can bad teaching induce forgetting? unlearning in deep networks using an incompetent teacher. In *AAAI Conference on Artificial Intelligence*, 2023.

Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 215–223, 2011.

Chongyu Fan, Jiancheng Liu, Yihua Zhang, Dennis Wei, Eric Wong, and Sijia Liu. SalUn: Empowering machine unlearning via gradient-based weight saliency. In *International Conference on Learning Representations (ICLR)*, 2024.

Jack Foster, Stefan Schoepf, and Alexandra Brintrup. Fast machine unlearning without retraining through selective synaptic dampening. In *AAAI Conference on Artificial Intelligence*, volume 38, pages 12043–12051, 2024.

Aditya Golatkar, Alessandro Achille, and Stefano Soatto. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9304–9312, 2020.

Aditya Golatkar, Alessandro Achille, Avinash Ravichandran, Marzia Polito, and Stefano Soatto. Forgetting outside the class: Scrubbing deep networks of information available in training data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

Laura Graves, Vineel Nagisetty, and Vijay Ganesh. Amnesiac machine learning. In *AAAI Conference on Artificial Intelligence*, 2021.

Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens Van Der Maaten. Certified data removal from machine learning models. In *International Conference on Machine Learning (ICML)*, pages 3832–3842, 2020.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009.

Meghdad Kurmanji, Peter Triantafillou, Jamie Hayes, and Eleni Triantafillou. Towards unbounded machine unlearning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011.

Quoc Phong Nguyen, Bryan Kian Hsiang Low, and Patrick Jaillet. Variational Bayesian unlearning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Gaurav Patel and Qiang Qiu. Learning to unlearn while retaining: Combating gradient conflicts in machine unlearning. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

Liwei Song and Prateek Mittal. Systematic evaluation of privacy risks of machine learning models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2615–2632, 2021.

Anvith Thudi, Gabriel Deza, Varun Chandrasekaran, and Nicolas Papernot. Unrolling SGD: Understanding factors of influence in machine unlearning. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 303–319, 2022.

Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. MedMNIST v2 – a large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10(1):41, 2023.

A Notation, Assumptions, and the Analysed Update

We restate the update exactly as implemented, since the entire point of Theorem 1 is that the analysed object and the executed object coincide.

Notation. $\widetilde{\mathcal{L}}_{\mathrm{r}}(\theta) = \mathcal{L}_{\mathrm{r}}(\theta) + \beta\mathcal{L}_{\mathrm{KD}}(\theta, \theta_0; \mathcal{D}_r)$ is the composite retain objective evaluated on the sampled retain mini-batch at step t, and $g_{\mathrm{r}} := \nabla_\theta \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_t)$ its gradient. Note β does not appear separately in the bounds because g_{r} is already the composite gradient. $g_{\mathrm{f}} := \nabla_\theta \mathcal{L}_{\mathrm{f}}(\theta_t; x_f)$ is the forget gradient on the forget mini-batch. $\lambda_{\mathrm{wd}} \geq 0$ is weight decay, $c > 0$ the clipping threshold, $\eta > 0$ the learning rate, and $\mu \in [0, 1)$ the momentum coefficient. $\|\cdot\|$ is the Euclidean norm on the concatenation of all trainable parameters.

The projection operator. For $g \neq 0$ define

$$\Pi_g(v) \;=\; \begin{cases} v, & \langle v, g\rangle \geq 0, \\ v - \dfrac{\langle v, g\rangle}{\|g\|^2}\, g, & \langle v, g\rangle < 0, \end{cases} \qquad \widehat{\Pi}_g(v) \;=\; \begin{cases} \Pi_g(v)\, \dfrac{\|v\|}{\|\Pi_g(v)\|}, & \Pi_g(v) \neq 0, \\ 0, & \Pi_g(v) = 0, \end{cases} \tag{8}$$

where $\widehat{\Pi}$ is the *magnitude-preserving* variant used by PURGE (§3.2). In the implementation the case $\Pi_g(v) = 0$ is detected by the guard $\|\Pi_g(v)\| \leq \epsilon\|v\|$ with $\epsilon = 10^{-8}$.

The analysed (and executed) update. With $v_{-1} = 0$, for $t = 0, 1, 2, \ldots$

$$d_t = \mathrm{clip}_c\big(g_{\mathrm{f}}^{(t)} + \lambda_{\mathrm{wd}}\theta_t\big), \qquad \text{where } \mathrm{clip}_c(x) = x \cdot \min\big(1, c/\|x\|\big), \tag{9}$$

$$v_t = \mu\, v_{t-1} + d_t, \tag{10}$$

$$u_t = \widehat{\Pi}_{g_{\mathrm{r}}^{(t)}}(v_t), \tag{11}$$

$$\theta_{t+1} = \theta_t - \eta\, u_t. \tag{12}$$

Three orderings are load-bearing and are stated here so they cannot be lost in implementation: clipping acts on d_t *before* the momentum accumulation (9)–(10); the projection acts on the accumulated v_t, i.e. on the update rather than on the gradient (11); and no clipping is applied after (11).

Assumptions.

(A1) $\widetilde{\mathcal{L}}_{\mathrm{r}}$ is L-smooth: $\|\nabla\widetilde{\mathcal{L}}_{\mathrm{r}}(x) - \nabla\widetilde{\mathcal{L}}_{\mathrm{r}}(y)\| \le L\|x - y\|$ for all x, y.

(A2) Clipping is applied as in (9), so $\|d_t\| \le c$ for all t.

(A3) $\mu \in [0, 1)$.

(A1) is the standard smoothness assumption of first-order analysis; for ReLU networks it holds piecewise, and we treat it as an idealisation, as is conventional. (A2) and (A3) are properties of the implementation, not assumptions about the data.

B Proof of Theorem 1

We prove three lemmas and then assemble the theorem.

Lemma 1 (Projection properties)**.** *Let $g \neq 0$ and v be arbitrary. Then*

(i) $\langle \Pi_g(v), g\rangle \ge 0$*, with equality whenever* $\langle v, g\rangle < 0$*;*

(ii) $\|\Pi_g(v)\| \le \|v\|$*;*

(iii) $\langle \widehat{\Pi}_g(v), g\rangle \ge 0$ *and* $\|\widehat{\Pi}_g(v)\| \le \|v\|$*, with* $\|\widehat{\Pi}_g(v)\| = \|v\|$ *whenever* $\Pi_g(v) \neq 0$*.*

Proof. (i) If $\langle v, g\rangle \ge 0$ then $\Pi_g(v) = v$ and the claim is immediate. If $\langle v, g\rangle < 0$ then

$$\langle \Pi_g(v), g\rangle = \Big\langle v - \tfrac{\langle v,g\rangle}{\|g\|^2} g,\ g \Big\rangle = \langle v, g\rangle - \tfrac{\langle v,g\rangle}{\|g\|^2}\|g\|^2 = 0.$$

(ii) In the first branch this is an equality. In the second branch, Π_g is the orthogonal projection onto the hyperplane $\{u : \langle u, g\rangle = 0\}$; writing $v = \Pi_g(v) + \frac{\langle v,g\rangle}{\|g\|^2} g$ as a sum of two orthogonal vectors, Pythagoras gives

$$\|v\|^2 = \|\Pi_g(v)\|^2 + \frac{\langle v, g\rangle^2}{\|g\|^2} \ \ge\ \|\Pi_g(v)\|^2. \tag{13}$$

(iii) If $\Pi_g(v) = 0$ then $\widehat{\Pi}_g(v) = 0$ and both claims are trivial. Otherwise $\widehat{\Pi}_g(v) = s\,\Pi_g(v)$ with $s = \|v\|/\|\Pi_g(v)\| > 0$, so $\langle \widehat{\Pi}_g(v), g\rangle = s\langle \Pi_g(v), g\rangle \ge 0$ by (i), and $\|\widehat{\Pi}_g(v)\| = s\|\Pi_g(v)\| = \|v\|$. □

Part (iii) is the formal content of the design choice in §3.2(iii): rescaling restores the magnitude that (13) removes, and because $s > 0$ it cannot flip the sign of the inner product. The guarantee is therefore invariant to the rescaling, while the step magnitude is not.

Lemma 2 (Velocity norm bound)**.** *Under (A2) and (A3), for every $t \ge 0$,*

$$\|v_t\| \ \le\ c\,\frac{1 - \mu^{\,t+1}}{1 - \mu} \ <\ \frac{c}{1 - \mu}. \tag{14}$$

Proof. Unrolling (10) from $v_{-1} = 0$ gives $v_t = \sum_{k=0}^{t} \mu^{\,t-k} d_k$. By the triangle inequality and (A2),

$$\|v_t\| \;\le\; \sum_{k=0}^{t} \mu^{\,t-k} \|d_k\| \;\le\; c \sum_{j=0}^{t} \mu^{\,j} \;=\; c\,\frac{1-\mu^{\,t+1}}{1-\mu} \;<\; \frac{c}{1-\mu},$$

the last step using $\mu \in [0,1)$ so that $\mu^{t+1} > 0$ and the geometric series converges. □

Lemma 2 is where the clip-before-momentum ordering is used: it requires $\|d_k\| \le c$ for the *ingredients* of the sum. Note also that the bound is attained only in the limit of perfectly aligned successive steps; we observe $\max_t \|u_t\| = 9.16$ against $c/(1-\mu) = 10$ in §5.4, so it is tight in practice.

Lemma 3 (Descent step)**.** *Under (A1), for any vector u and $\theta_+ = \theta_t - \eta u$,*

$$\widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_+) \;\le\; \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_t) - \eta \langle u, g_{\mathrm{r}} \rangle + \tfrac{1}{2} L \eta^2 \|u\|^2. \tag{15}$$

Proof. Standard consequence of L-smoothness (the descent lemma): (A1) implies $\widetilde{\mathcal{L}}_{\mathrm{r}}(y) \le \widetilde{\mathcal{L}}_{\mathrm{r}}(x) + \langle \nabla \widetilde{\mathcal{L}}_{\mathrm{r}}(x), y - x \rangle + \frac{L}{2}\|y-x\|^2$ for all x, y. Substituting $x = \theta_t$, $y = \theta_+$, so that $y - x = -\eta u$ and $\nabla \widetilde{\mathcal{L}}_{\mathrm{r}}(x) = g_{\mathrm{r}}$, gives (15). □

Theorem 3 (Theorem 1, restated)**.** *Under (A1)–(A3), the update (9)–(12) satisfies, at every step t and unconditionally,*

$$\textit{(a)} \quad \langle u_t, g_{\mathrm{r}}^{(t)} \rangle \;\ge\; 0, \tag{16}$$

$$\textit{(b)} \quad \|u_t\| \;\le\; \|v_t\| \;<\; \frac{c}{1-\mu}, \quad \textit{with } \|u_t\| = \|v_t\| \textit{ unless } \Pi_{g_{\mathrm{r}}}(v_t) = 0, \tag{17}$$

$$\textit{(c)} \quad \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_{t+1}) \;\le\; \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_t) + \tfrac{1}{2} L \eta^2 \Big(\frac{c}{1-\mu} \Big)^2. \tag{18}$$

Proof. (a) Immediate from Lemma 1(iii) applied to v_t with $g = g_{\mathrm{r}}^{(t)}$, using (11). If $g_{\mathrm{r}}^{(t)} = 0$ the projection is undefined and is skipped; then $\langle u_t, g_{\mathrm{r}}^{(t)} \rangle = 0$ and (a) holds trivially (see Remark 2).

(b) Combine Lemma 1(iii) with Lemma 2.

(c) By Lemma 3 with $u = u_t$,

$$\widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_{t+1}) \le \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_t) - \eta \underbrace{\langle u_t, g_{\mathrm{r}}^{(t)} \rangle}_{\ge 0 \text{ by (a)}} + \tfrac{1}{2} L \eta^2 \underbrace{\|u_t\|^2}_{< (c/(1-\mu))^2 \text{ by (b)}} \;\le\; \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_t) + \tfrac{1}{2} L \eta^2 \Big(\frac{c}{1-\mu} \Big)^2.$$ □

The first-order change in the sampled retain objective is thus non-positive at every step; the only admissible increase is the $O(\eta^2)$ curvature term. The guarantee is *unconditional* in the sense that it requires no assumption on g_{f}, on the alignment between forget and retain gradients, or on the data.

Corollary 1 (Momentum-free reduction)**.** *Setting $\mu = 0$ gives $v_t = d_t$, $\|u_t\| \le c$, and $\widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_{t+1}) \le \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_t) + \frac{1}{2} L \eta^2 c^2$, recovering the momentum-free per-step bound.*

Corollary 2 (A-GEM reduction)**.** *Setting $\mu = 0$, $c = \infty$ and letting $\eta \to 0$ recovers the first-order A-GEM guarantee $\langle u_t, g_{\mathrm{r}} \rangle \ge 0$ [Chaudhry et al., 2019].*

Corollary 1 is worth reading alongside Table 6: the $\mu = 0$ bound is *formally tighter* (by a factor $(1-\mu)^{-2} = 100$ at $\mu = 0.9$), yet the configuration it describes fails to unlearn on five of six datasets. A tighter bound on a method that does not forget is not the better result—which is precisely why we analyse the update that is actually run.

B.1 Why the Projection Must Be Applied to the Update

The following proposition shows that the design choice of §3.2(i) is not a matter of convenience: projecting the *gradient* provides no per-step guarantee once $\mu > 0$, however carefully each individual gradient is projected.

Proposition 1 (Gradient-projection is insufficient under momentum)**.** *Consider the variant that projects the gradient and then accumulates momentum:*

$$\tilde{d}_t = \mathrm{clip}_c\big(\Pi_{g_{\mathrm{r}}^{(t)}}(g_{\mathrm{f}}^{(t)})\big), \qquad v_t = \mu v_{t-1} + \tilde{d}_t, \qquad \theta_{t+1} = \theta_t - \eta v_t,$$

so that every accumulated ingredient satisfies $\langle \tilde{d}_t, g_{\mathrm{r}}^{(t)} \rangle \geq 0$*. Then for every* $\mu \in (0,1)$ *there exist retain and forget gradients for which the applied update* v_t *satisfies* $\langle v_t, g_{\mathrm{r}}^{(t)} \rangle < 0$*, i.e. the step strictly increases the sampled retain loss to first order.*

Proof. Work in $\mathbb{R}^2$ with $\lambda_{\mathrm{wd}} = 0$, $c = \infty$ (clipping inactive), and $v_{-1} = 0$.

Step 0. Let $g_{\mathrm{r}}^{(0)} = (1,0)$ and $g_{\mathrm{f}}^{(0)} = (-1,1)$. Then $\langle g_{\mathrm{f}}^{(0)}, g_{\mathrm{r}}^{(0)} \rangle = -1 < 0$, so the projection fires:

$$\tilde{d}_0 = \Pi_{g_{\mathrm{r}}^{(0)}}(g_{\mathrm{f}}^{(0)}) = (-1,1) - \frac{-1}{1}(1,0) = (0,1),$$

which is retain-safe, $\langle \tilde{d}_0, g_{\mathrm{r}}^{(0)} \rangle = 0$. Hence $v_0 = (0,1)$.

Step 1. Let the retain mini-batch change so that $g_{\mathrm{r}}^{(1)} = (0,-1)$, and let $g_{\mathrm{f}}^{(1)} = (1,0)$. Then $\langle g_{\mathrm{f}}^{(1)}, g_{\mathrm{r}}^{(1)} \rangle = 0 \geq 0$, so no projection is needed and $\tilde{d}_1 = (1,0)$, again retain-safe: $\langle \tilde{d}_1, g_{\mathrm{r}}^{(1)} \rangle = 0$. The applied update is

$$v_1 = \mu v_0 + \tilde{d}_1 = \mu(0,1) + (1,0) = (1,\mu), \qquad \langle v_1, g_{\mathrm{r}}^{(1)} \rangle = \langle (1,\mu), (0,-1) \rangle = -\mu \ < \ 0$$

for every $\mu > 0$. By Lemma 3 the first-order change in $\widetilde{\mathcal{L}}_{\mathrm{r}}$ is $-\eta \langle v_1, g_{\mathrm{r}}^{(1)} \rangle = +\eta\mu > 0$: the retain loss strictly increases to first order, even though *every* projected gradient entering the buffer was individually retain-safe. □

The mechanism is transparent: the momentum buffer carries a direction that was certified against $g_{\mathrm{r}}^{(0)}$ into a step whose constraint is $g_{\mathrm{r}}^{(1)}$, and the projection at step 1 never inspects the carried term. Projecting v_t itself (11) removes exactly this failure, because the certificate is issued against the constraint in force at the step where the update is applied. Note also that μ appears linearly in the violation, so the defect grows with the momentum coefficient rather than vanishing.

B.2 Why Clipping Must Precede Accumulation

Proposition 2 (Clipping the update destroys momentum amplification)**.** *Consider the variant that clips the final update,* $u_t = \mathrm{clip}_c(\widehat{\Pi}_{g_{\mathrm{r}}}(\mu v_{t-1} + \tilde{g}_t))$ *with* $\tilde{g}_t$ *unclipped. Then* $\|u_t\| \leq c$ *for all* t*, identical to the bound obtained at* $\mu = 0$ *(Corollary 1), irrespective of* μ*.*

Proof. Immediate from the definition of clip_c. □

Proposition 2 is trivial as mathematics but decisive in practice. In our runs the raw forget-gradient norm is $\sim$100$\times$ the clipping threshold ($\|g_{\mathrm{f}}\| \in [102, 149]$ with $c = 1$ on CIFAR-10), so clipping is active at every step and the cap is always attained. Clipping after accumulation therefore yields an applied step of magnitude exactly ηc — the same magnitude as the momentum-free configuration, and hence the same failure to forget documented in Table 6. The ordering in (9)–(10) instead admits $\|v_t\| \to c/(1-\mu)$, a 10$\times$ larger step at $\mu = 0.9$, while Lemma 2 keeps it bounded. The bound in (18) is looser by $(1-\mu)^{-2}$ as a result; this is the price of a guarantee that describes an optimiser that unlearns.

B.3 Remarks on Scope and Exactness

Remark 1 (Mini-batch scope). $g_{\mathrm{r}}^{(t)}$ is the gradient of the retain objective on the *sampled* retain mini-batch, so (18) is exact for that mini-batch's objective—which is the quantity PURGE projects against, and therefore the quantity about which a per-step statement can be made. Because mini-batch gradients are unbiased estimates of the population gradient, the first-order term is non-positive in expectation up to the standard SGD gradient-variance term; a population-exact per-step guarantee would require the full retain gradient at every step, at $O(|\mathcal{D}_r|)$ cost per update. We regard this as the principal formal limitation (§7).

Remark 2 (Degenerate retain gradient). The implementation skips the projection when $\|g_{\mathrm{r}}\|^2 < 10^{-12}$, since (8) is then numerically ill-posed. In that regime the violation of (16) is bounded by Cauchy–Schwarz: $|\langle u_t, g_{\mathrm{r}}\rangle| \le \|u_t\|\|g_{\mathrm{r}}\| \le 10^{-6}c/(1-\mu)$, giving a first-order retain increase of at most $10^{-5}\eta$. Moreover such a g_{r} means the retain mini-batch is already at a stationary point. The exception is therefore quantitatively negligible, but we state it rather than suppress it.

Remark 3 (Floating-point exactness). In exact arithmetic $\langle \Pi_g(v), g\rangle = 0$ identically. In float32, the inner products are sums over 1.12×10^7 parameters, with accumulation error of order $\sqrt{n}\,\varepsilon_{32} \approx 4 \times 10^{-4}$ relative. The measured worst-case relative violation is -7.4×10^{-7} (§5.4), three orders of magnitude *below* that scale and with positive mean, consistent with rounding rather than a systematic defect. We therefore state Theorem 3 as holding to float32 precision, and recommend float64 accumulation for the inner products in (8) in reference implementations.

C Proof of Theorem 2

Theorem 4 (Theorem 2, restated)**.** *Fix θ_0 with retain gradient $g_{\mathrm{r}} := \nabla\widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_0) \neq 0$ and forget gradient g_{f}. Consider one-step update operators. Then:*

(i) ***GA.*** *The update $\theta_0 + \eta g_{\mathrm{f}}$ strictly increases $\widetilde{\mathcal{L}}_{\mathrm{r}}$ to first order whenever $\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle > 0$.*

(ii) ***SCRUB.*** *For any fixed $\lambda > 0$ there exist $g_{\mathrm{f}}, g_{\mathrm{r}}$ with $\langle -g_{\mathrm{f}} + \lambda g_{\mathrm{r}},\ g_{\mathrm{r}}\rangle < 0$.*

(iii) ***SalUn.*** *The masked random-label update $-M \odot \nabla \mathrm{CE}_{\mathrm{RL}}$ has unconstrained sign with respect to g_{r}.*

(iv) ***Soft conflict regularisers.*** *Penalising negative alignment does not preclude it.*

(v) ***Purge.*** *$\langle u_t, g_{\mathrm{r}}\rangle \ge 0$ unconditionally.*

Proof. (i) A first-order expansion gives $\widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_0 + \eta g_{\mathrm{f}}) = \widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_0) + \eta\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle + O(\eta^2)$, which exceeds $\widetilde{\mathcal{L}}_{\mathrm{r}}(\theta_0)$ for small $\eta > 0$ when $\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle > 0$. Since GA constrains neither the sign nor the magnitude of $\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle$, no configuration-independent per-step guarantee is available.

(ii) Compute $\langle -g_{\mathrm{f}} + \lambda g_{\mathrm{r}},\ g_{\mathrm{r}}\rangle = -\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle + \lambda\|g_{\mathrm{r}}\|^2$. This is negative precisely when $\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle > \lambda\|g_{\mathrm{r}}\|^2$. Such a pair exists for any fixed λ: e.g. in $\mathbb{R}^2$ take $g_{\mathrm{r}} = (1, 0)$ and $g_{\mathrm{f}} = (\lambda + 1)\,(1, 0)$, giving $\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle = \lambda + 1 > \lambda = \lambda\|g_{\mathrm{r}}\|^2$. Because λ is a fixed hyperparameter while $\langle g_{\mathrm{f}}, g_{\mathrm{r}}\rangle / \|g_{\mathrm{r}}\|^2$ varies along the trajectory, no fixed λ certifies every step.

(iii) With random labels the masked gradient direction is uncorrelated with g_{r} in expectation; individual draws therefore realise either sign of $\langle g_{\mathrm{SalUn}}, g_{\mathrm{r}}\rangle$, and the mask M (selected by gradient magnitude, not by retain alignment) imposes no constraint on it.

(iv) A regulariser adding $\rho\,\phi(\langle g, g_{\mathrm{r}}\rangle)$ to the objective with ϕ decreasing shifts the optimum toward positive alignment but leaves the feasible set unrestricted: for any finite penalty weight ρ, a sufficiently large forget-loss reduction makes a step with $\langle g, g_{\mathrm{r}}\rangle < 0$ preferable. Only a hard constraint—i.e. a projection onto $\{u : \langle u, g_{\mathrm{r}}\rangle \ge 0\}$—renders such steps unreachable.

(v) Theorem 3(a). □

Remark 4. Theorem 4 is a *worst-case* separation: it exhibits configurations in which the baselines strictly degrade the retain objective in a single step, not a claim that they typically do. In practice all these methods often produce small retain-loss changes. The distinction the theorem draws is between behaviour that is empirically usual and behaviour that is guaranteed by construction; only the latter is checkable without running the method.

D Per-Class Results

The main text reports per-dataset aggregates (Table 3). Because a mean over classes can conceal a method that succeeds on easy classes and fails on hard ones, we report every class individually. Each row is the mean ± standard deviation over seeds $\{42, 123, 456\}$. CIFAR-100 per-class results appear in the main text (Table 5); the raw per-run CSVs for all 75 runs are released with the code.

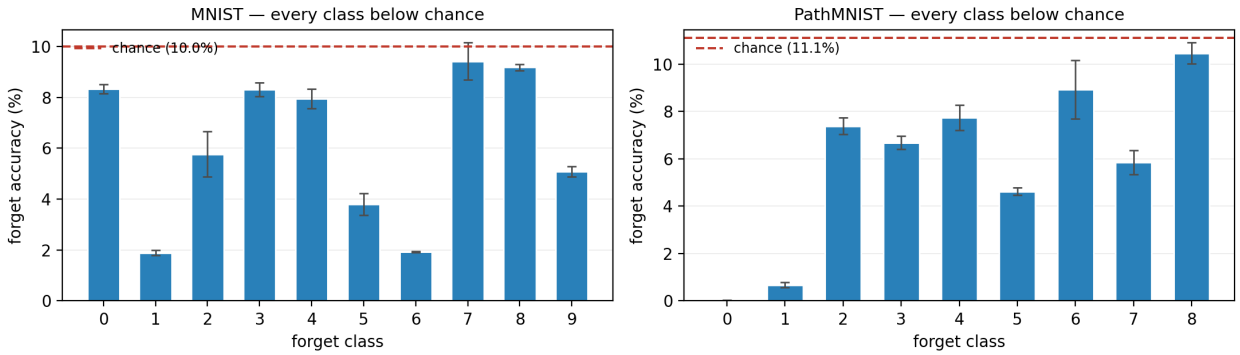


Figure 12: **Per-class forget accuracy** (mean ± std over three seeds). Every MNIST and PathMNIST class individually falls below its chance level, so the per-dataset means of Table 3 are not concealing a class that fails.

Table 18: **MNIST per-class forgetting** (mean ± std over 3 seeds). Chance level is 10.0%; every class attains FA below it. TA is measured on the full test set, so it necessarily drops by ∼10pp relative to the 99% base model once the forget class is correctly no longer recognised.

Forget class	TA (%)	FA (%)↓	RA (%)	MIA
0	90.26 ± 0.05	8.33 ± 0.18	99.42 ± 0.00	0.495 ± 0.002
1	88.29 ± 0.01	1.89 ± 0.09	99.50 ± 0.02	0.512 ± 0.001
2	89.68 ± 0.07	5.77 ± 0.89	99.51 ± 0.01	0.491 ± 0.001
3	90.37 ± 0.02	8.31 ± 0.27	99.45 ± 0.01	0.483 ± 0.001
4	90.22 ± 0.01	7.95 ± 0.39	99.43 ± 0.02	0.510 ± 0.001
5	90.76 ± 0.04	3.80 ± 0.43	99.52 ± 0.03	0.499 ± 0.001
6	90.08 ± 0.02	1.92 ± 0.03	99.50 ± 0.02	0.501 ± 0.001
7	90.03 ± 0.03	9.42 ± 0.74	99.51 ± 0.01	0.490 ± 0.000
8	90.36 ± 0.03	9.19 ± 0.13	99.50 ± 0.02	0.486 ± 0.001
9	89.97 ± 0.05	5.09 ± 0.21	99.54 ± 0.02	0.503 ± 0.000
Mean	**90.00**	**6.17**	**99.49**	**0.497**

D.1 MNIST (all 10 classes)

Table 18 gives all ten MNIST classes. Every class individually falls below the 10% chance level, with per-class mean FA ranging from 1.89% (class 1) to 9.42% (class 7); the aggregate is therefore not concealing a failure. Retain accuracy is essentially constant across classes (99.42–99.54%), and MIA stays within 0.017 of the ideal 0.5 for every class. Seed variance is small throughout (the largest FA standard deviation across classes is 0.89pp), indicating the procedure is stable rather than seed-lucky.

D.2 PathMNIST (all 9 classes)

Table 19 gives all nine PathMNIST tissue classes. Every class again falls below the 11.1% chance level, from 0.02% (class 0) to 10.46% (class 8). Two observations deserve stating explicitly rather than being averaged away.

First, the spread is far wider than on MNIST—per-class TA ranges from 73.23% to 89.28%—reflecting genuinely unequal class difficulty in histopathology rather than instability: within-class seed variance remains small (largest FA standard deviation 1.23pp).

Second, and more importantly, **MIA AUROC is not uniformly near** 0.5: class 2 reaches 0.298 and class 3 0.388, both well *below* the ideal. A value substantially below 0.5 indicates that forget-class training samples have become *harder* to classify than forget-class test samples—the signature of *over*-forgetting, in

Table 19: **PathMNIST per-class forgetting** (mean ± std over 3 seeds). Chance level is 11.1%; every class attains FA below it. †Classes 2 and 3 show MIA well below the ideal 0.5, indicating over-forgetting (see text).

Forget class	TA (%)	FA (%)↓	RA (%)	MIA
0	73.23 ± 0.06	0.02 ± 0.02	99.01 ± 0.01	0.511 ± 0.003
1	78.66 ± 0.26	0.68 ± 0.12	98.48 ± 0.08	0.473 ± 0.003
2†	89.28 ± 0.12	7.38 ± 0.35	99.02 ± 0.02	0.298 ± 0.003
3†	82.40 ± 0.12	6.68 ± 0.28	98.86 ± 0.02	0.388 ± 0.005
4	78.04 ± 0.08	7.73 ± 0.53	98.95 ± 0.03	0.502 ± 0.001
5	84.58 ± 0.03	4.62 ± 0.15	98.32 ± 0.02	0.548 ± 0.001
6	78.58 ± 0.25	8.93 ± 1.23	98.77 ± 0.06	0.541 ± 0.001
7	88.48 ± 0.14	5.85 ± 0.50	99.25 ± 0.01	0.562 ± 0.001
8	75.36 ± 0.04	10.46 ± 0.45	99.28 ± 0.03	0.525 ± 0.001
Mean	**80.96**	**5.82**	**98.88**	**0.483**

which the unlearned model is distinguishable from a retrained one in the opposite direction to a privacy leak. It is detectable precisely because we report MIA per class and treat distance from 0.5 in either direction as a defect. We flag this as a real limitation of the current stopping rule on PathMNIST: dual stopping targets FA, which does not constrain how far past the retrained model's output statistics the optimiser travels.

On the per-dataset MIA aggregate. The PathMNIST aggregate of 0.483 ± 0.084 in Table 3 is within 0.017 of ideal, but its standard deviation is an order of magnitude larger than any other dataset's precisely because of classes 2 and 3. The aggregate is accurate but uninformative here; this is a concrete instance of the more general point of §5 that MIA must be read alongside FA, and per-class rather than only in aggregate.

D.3 BN-robust Purge on all ten CIFAR-10 classes

Table 20 gives every CIFAR-10 class under the recalibration attack of §5.6, one seed per class (class 0 has three seeds in Table 10). "Pre" and "post" are FA under test-time transforms before and after `update_bn` on the retain set. All runs share the 93.97% base of §5.6. The calibrated runs here read the stopping FA on the full retain set rather than the 10,240-image subset; on class 0 the two made identical stopping decisions (§5.6).

E Hyperparameter Configuration

Table 21 lists the settings shared across all datasets and Table 22 the per-dataset settings. All are released as YAML configuration files; the reported runs use them unmodified.

Tuning effort. Only four quantities vary across datasets: the epoch cap T, the learning rate η, the KD weight β, and the retain budget α. The learning rate takes one of two values (5×10^{-4} for the CIFAR datasets, 2×10^{-4} elsewhere), β one of three, and α one of two. The forget target ϵ_F is not tuned at all: it is fixed to the chance level $100/K$ implied by the task. Notably, μ, c, λ_{wd}, and T_{KD}—the four quantities that enter the guarantee of Theorem 3—are held constant across all six datasets, so the bound $\frac{1}{2}L\eta^2(c/(1-\mu))^2$ varies across datasets only through η (and through the dataset-dependent smoothness constant L).

A note on defaults. The command-line default for `-max_grad_norm` in the released code is 5.0, whereas every configuration file sets $c = 1.0$. Runs launched without `-config` would therefore use a different clip threshold, and a correspondingly weaker constant in (18). All results in this paper use the configuration

Table 20: **BN-robust PURGE per class** (CIFAR-10, seed 42). The frozen-statistics configuration is recovered on every class. BN-robust is not, under either stopping rule, and with calibrated stopping every class ships within three standard errors of chance (11.3%). Epochs "25→22": the run reached the cap and its best epoch was shipped.

	Frozen		BN-robust, naive		BN-robust, calibrated				
Class	pre	post	pre	post	pre	post	epochs	TA	RA
0	8.76	99.22	9.24	9.28	9.22	9.24	13	84.35	95.60
1	2.40	97.60	10.66	10.66	6.66	6.66	8	84.56	96.19
2	6.52	98.30	11.18	11.16	9.90	9.92	15	85.07	95.27
3	4.78	94.82	11.30	11.30	10.00	9.98	16	85.92	96.26
4	4.84	97.34	13.32	13.34	10.18	10.16	25→22	84.88	94.74
5	7.28	94.70	8.78	8.78	9.40	9.42	12	84.52	94.64
6	9.18	99.14	7.20	7.20	7.30	7.30	8	83.32	94.41
7	3.38	96.20	11.36	11.34	9.12	9.12	10	84.49	95.24
8	4.54	99.26	16.74	16.68	9.74	9.68	14	83.59	94.67
9	2.52	98.16	15.44	15.42	8.92	8.92	11	83.84	95.01
Mean	5.42	97.47	11.52	11.52	9.04	9.04	—	84.45	95.20

Table 21: **Shared hyperparameters** (identical for all six datasets). The two components removed from PURGE are listed explicitly with value 0, since they remain available as flags for the ablation of §5.8.

Symbol	Flag	Value	Role
μ	`-momentum`	0.9	momentum; projection applied *after* accumulation
c	`-max_grad_norm`	1.0	clip on d_t, *before* accumulation
λ_{wd}	`-weight_decay`	5×10^{-4}	weight decay, applied inside d_t
T_{KD}	`-temperature`	4.0	distillation temperature
—	`-forget_objective`	`kl_retain`	KL toward retain-confusion p_r
—	`-batch_size`	128	forget and retain mini-batch size
—	`-fa_check_freq`	50	intra-epoch FA check interval (batches)
—	`-seed`	$\{42, 123, 456\}$	three seeds per task
Removed components (retained as ablation flags only)			
λ	`-rep_weight`	0.0	representation erasure (removed)
γ	`-forget_gate`	0.0	entropy gate (removed)

files. We note this because the value of c enters the guarantee directly, so it should not be inherited silently from a default.

F Distance to the Retrained Model: A Refuted Hypothesis

This appendix reports a negative result. We formed a hypothesis about why PURGE over-forgets on two PathMNIST classes, derived a deployable fix from it, pre-committed to a falsifiable prediction, ran the experiment, and the prediction did not hold. We report it in full because the experiment also produced the diagnosis that we believe is correct, and because the refuted hypothesis concerns PURGE's own forget objective.

F.1 The defect, and the hypothesis

Per-class reporting (Appendix D) showed that on PathMNIST classes 2 and 3, MIA AUROC reaches 0.298 and 0.388—far *below* the ideal 0.5. Forget accuracy is satisfied on both, so FA alone certifies these runs as successes when they are not. A value below 0.5 means forget-class training samples have become *harder* to

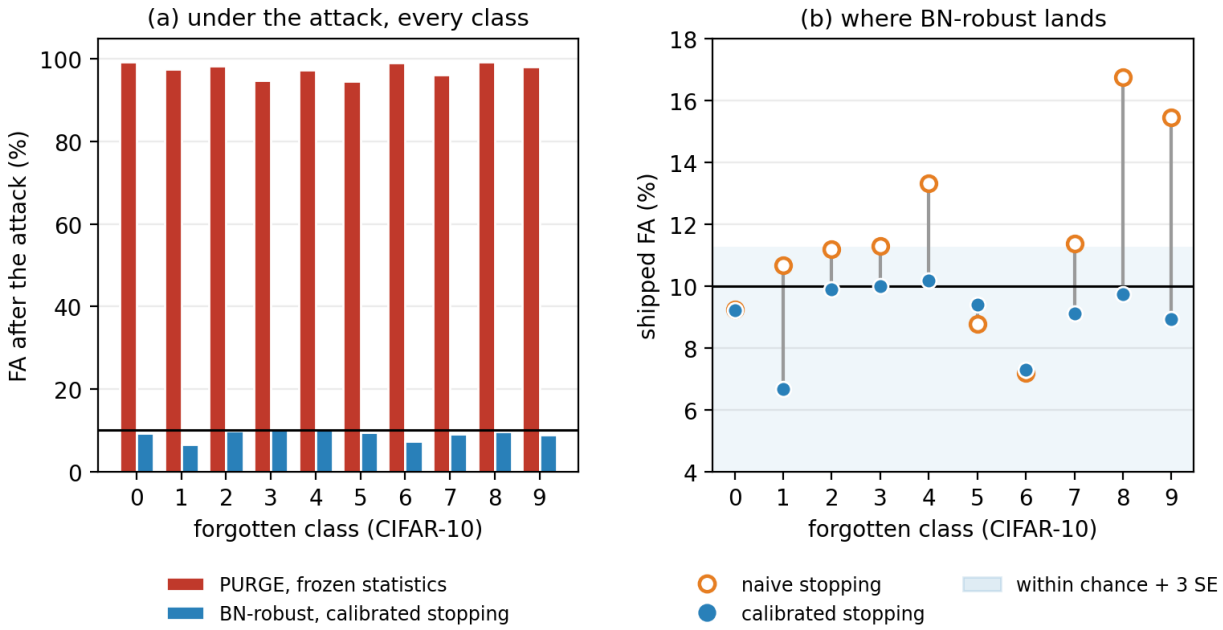


Figure 13: **BN-robust on all ten CIFAR-10 classes** (seed 42; the data of Table 20). (a) Under the attack, the frozen-statistics configuration is recovered on every class and BN-robust on none. (b) Where BN-robust ships: with naive stopping (open) five classes land above chance + 3 SE; with calibrated stopping (filled), none do.

classify than forget-class test samples: the model is still distinguishable from a retrained one, in the opposite direction to a privacy leak.

Our forget-side constraint (Eq. 2) is $\mathrm{KL}(p_\theta(x_f) \,\|\, p_r) \le \epsilon_F$, a statement about *distributions*, but dual stopping (§3.5) implements it as FA $\le \epsilon_F$, a statement about *argmax*. These are not equivalent, and we hypothesised that the discrepancy was the cause: FA cannot distinguish (a) $p_\theta(x_f)$ having moved *to* the distribution a retrained model produces from (b) having moved *past* it, since both give FA ≈ 0.

The hypothesised fix was to enforce the constraint as stated. Since p_r is computed from the *original* model, this needs no retrained reference and is deployable: stop on $\mathrm{KL}(p_\theta(x_f)\|p_r)$ rather than on FA. We reasoned that this quantity is minimised at $p_\theta = p_r$ and rises on either side, so a stop-when-it-stops-improving rule would halt at the minimum—exactly the point FA cannot locate.

Prediction (pre-committed). Replacing FA stopping with KL stopping would, on the affected classes, (1) reduce $|\mathrm{MIA} - 0.5|$, (2) reduce Jensen–Shannon divergence to a retrained reference, and (3) not materially worsen FA or RA, while leaving an unaffected control class approximately unchanged.

F.2 Design

We retrained reference models without the forget class for PathMNIST classes 2 and 3 (affected) and class 4 (control, MIA already 0.501), keeping all K output heads so that output distributions remain comparable, and training all three identically for a fixed 30-epoch cosine schedule so that no reference is better converged than another. Arm A is PURGE with FA stopping (the deployed configuration); arm B disables FA stopping and enables KL stopping. Oracle metrics are computed against the references: JS divergence between output distributions on the forget set (primary), top-1 agreement with the reference, Fréchet distance between penultimate features, and MIA.

The control class is the load-bearing part of the design: a criterion that merely stops earlier everywhere would improve the affected classes while also perturbing the control, whereas a targeted fix should leave the control alone.

Table 22: **Per-dataset hyperparameters.** K is the number of classes. Note that the forget-accuracy target ϵ_F is set to the chance level $100/K$ for every dataset—it is not tuned per dataset, but derived from the task.

Dataset	K	Epochs T	η	β (KD)	α (retain budget)	$\epsilon_F = 100/K$
CIFAR-10	10	15	5×10^{-4}	2.0	5.0	10.0%
CIFAR-100	100	20	5×10^{-4}	2.0	5.0	1.0%
MNIST	10	12	2×10^{-4}	3.0	4.0	10.0%
SVHN	10	15	2×10^{-4}	3.0	5.0	10.0%
STL-10	10	15	2×10^{-4}	5.0	5.0	10.0%
PathMNIST	9	12	2×10^{-4}	5.0	4.0	11.1%

Table 23: **KL stopping vs. FA stopping on PathMNIST**, measured against retrained references. Arrows mark the direction of improvement. The prediction was that the affected classes would improve and the control would not move; neither held.

Class	Arm	$\lvert\text{MIA}-0.5\rvert \downarrow$	JS↓	agree %↑	Fréchet↓	KL to p_r
2 (affected)	FA stop	0.205	0.308	45.4	185.7	0.957
	KL stop	0.268	0.412	23.5	332.0	0.052
3 (affected)	FA stop	0.115	0.382	31.2	375.2	0.805
	KL stop	**0.068**	0.421	**39.8**	780.1	0.002
4 (control)	FA stop	0.001	0.286	38.7	141.1	1.510
	KL stop	0.044	0.399	12.3	297.7	0.116

F.3 Result: the prediction fails

Table 23 reports the outcome. Across three classes and four metrics, arm B is better on 2 comparisons and worse on 10. The primary oracle metric—JS divergence to the retrained model—is *worse in all three classes*. The control class degrades on every metric, which is precisely the signature that rules out a targeted fix. Only class 3 improves on $|\text{MIA} - 0.5|$.

The proxy is anti-correlated with the oracle. A proxy is only usable if it tracks the quantity it stands in for. Across the six checkpoints, the rank correlation between $\text{KL}(p_\theta \| p_r)$ and JS divergence to the retrained model is $\rho = -1.00$ (Pearson -0.94): the ordering is perfectly inverted. Driving the deployable proxy toward zero moves the model monotonically *away* from the retrained reference. This is the opposite of the relationship the hypothesis required.

F.4 Why it failed

(i) The criterion never fired. In all three runs of arm B the terminating condition was the retain-loss budget, not the KL rule. The reason is visible in the logged trajectories: $\text{KL}(p_\theta(x_f) \| p_r)$ decreases essentially monotonically (class 2: $5.53 \to 0.05$ over 8 checks with zero increases; class 4: $2.73 \to 0.07$, zero increases; class 3: $2.06 \to 0.002$ over 20 checks with 4 small increases). It never turns back up, so a stop-when-it-stops-improving rule cannot trigger. The U-shape the hypothesis assumed does not exist over the range the optimiser traverses.

This is also a limitation of the test, and we state it plainly: because the KL rule never fired and FA stopping was disabled, arm B is in effect PURGE *without forget-side stopping* (FA reaches 0.0 in all three runs), which is closer to the −Stopping ablation than to a KL-stopped method. The comparison in Table 23 is therefore evidence against the criterion *as specified*, not a clean contrast between two stopping rules.

(ii) p_r is a global average; a retrained model routes per-sample. p_r is a single distribution, the retain-set average softmax, and it is the same target for every forget sample. A retrained model does something different: it routes each forget-class image to whichever surviving class that particular image most resembles, giving an input-dependent output distribution. Matching p_r more closely therefore makes the model's predictions on forget data unnaturally homogeneous across samples. The agreement column is the evidence: as KL to p_r falls by one to three orders of magnitude, top-1 agreement with the reference *collapses* on two of the three classes (45.4 → 23.5 on class 2, 38.7 → 12.3 on class 4). If p_r were the right target, matching it better should have raised agreement. Class 3 moves the other way (31.2 → 39.8), so the effect is not universal.

Confound we cannot separate. KL to p_r decreases with training time while JS to the reference increases over the observed range, so the two are mechanically anti-correlated and the $\rho = -1.00$ figure partly reflects run duration rather than target quality alone. The agreement collapse is the part of the evidence that speaks to the target itself, and it rests on two of three classes. We therefore claim the criterion is refuted, and treat the mechanism as well-supported but not established.

F.5 What the experiment did establish

The FA rule overshoots its own target. All three FA-stopped runs halt at FA ≈ 7% against a target of 11.1% (the chance level), overshooting by 3.9–4.2pp, because the intra-epoch FA check is evaluated only every 50 batches. The over-forgetting we set out to explain is thus at least partly a *granularity* defect in how the existing criterion is enforced, not evidence that FA is the wrong quantity.

Less unlearning was better on every class. JS divergence to the retrained model is lower at the earlier stopping point in all three classes. Over the range we observe, the model is still moving toward the retrained reference when FA stopping halts it, and continues away from it afterwards. Combined with the overshoot above, this points to stopping *earlier and more precisely* rather than to replacing the stopped quantity.

Consequence for the method. We draw a narrower conclusion about `kl_retain` than the original formulation implies. As a forget-side *direction*, p_r is a better target than the uniform distribution (§3.3). As a *target to be minimised to zero*, it is actively harmful. Accordingly the constraint $\mathrm{KL}(p_\theta(x_f)\|p_r) \leq \epsilon_F$ in Eq. 2 should be read as a tolerance that is satisfied early, not as an objective to be driven to its minimum, and we do not claim p_r is what a retrained model produces on forget inputs—the measurements here show it is not.

F.6 Open question

Where JS divergence to the retrained model is minimised along the unlearning trajectory remains unmeasured. Since JS is still falling when FA stopping fires and rising afterwards, a minimum exists between the base model and the FA-stopped point; locating it would establish whether a single FA level identifies it across classes, which would make it deployable, or whether it varies per class, which would not. This question is answered in §5.14: the minimum is interior, lies above the chance level, and the current rule stops past it.

G Ethical and Societal Considerations

Unlearning as evidence removal. A method that makes a model indistinguishable from one retrained without specific data could be misused to destroy evidence that problematic data was used for training. This is a dual-use property of the unlearning goal itself rather than of PURGE specifically, but PURGE does make it concrete. Partial mitigation is that PURGE leaves an audit trail by construction: projection-rate logs, the FA trajectory, and retain-loss budget history are all recorded per run, so an auditor with access to the logs can establish that unlearning was applied and to what extent. We note this is mitigation, not prevention—an adversary who controls the logs controls the audit.

Selective fairness degradation. Class-level unlearning removes all samples of a class, which may correspond to a demographic group. Crucially, the accuracy degradation we observe is *not* confined to the forgotten class: test accuracy falls beyond what the removed class alone accounts for, and the per-class spread on PathMNIST (Appendix D) shows the effect is uneven across classes. Deployers should therefore monitor per-class accuracy before and after unlearning rather than only aggregate accuracy, which can conceal a disproportionate cost borne by one group.

Over-forgetting as a distinct harm. The PathMNIST finding (Appendix D) shows that a model can be pushed *past* the retrained reference. This matters ethically as well as technically: such a model is still distinguishable from one trained without the data, so a deletion request has not been faithfully honoured, yet every FA-based check reports success. Compliance evidence resting on forget accuracy alone would be misleading here.

Regulatory gap. Approximate unlearning methods, including PURGE, do not satisfy formal (ε, δ)-differential-privacy certification. Whether MIA AUROC ≈ 0.5 constitutes legally sufficient "erasure" under instruments such as the GDPR right to erasure remains an open legal question on which we take no position. The per-step guarantee of Theorem 1 is a statement about *retain-side utility*, not a privacy certificate, and should not be presented as one.

H Reproducibility

Artefacts. Code, YAML configuration files, per-run summary CSVs for all 75 runs, and the BatchNorm-recalibration diagnostic are released in the anonymised repository. **[TODO: Insert the anonymised repository URL.]** The archived $\mu = 0$ results underlying Table 6 are released alongside the main results, so the momentum ablation can be re-derived without re-running it.

Environment. PyTorch $\geq$ 2.0 with CUDA; ResNet-18 (11.7M parameters) throughout. Runs were executed on a single GPU. Datasets download automatically except PathMNIST, which is obtained through `medmnist`.

Implementation details that affect results.

- Momentum and weight decay are applied *manually* so that the projection acts on the post-momentum update; the underlying `torch` SGD object is configured with `momentum=0`, `weight_decay=0` and acts only as a plain step. Reading the optimiser construction alone is therefore misleading—see §3.2 and Appendix A.
- Clipping is applied to d_t before momentum accumulation, and *not* to the final update. Reversing this changes the effective step size by a factor $1/(1-\mu)$ (Proposition 2).
- The projected update is rescaled to the pre-projection norm.
- BatchNorm layers are set to `eval()` immediately after the model is set to `train()`, so running statistics are frozen during unlearning. BN-robust PURGE (§5.6) instead recomputes them from 512 retain images before every step (`-bn_robust exact -bn_recal_samples 512`) and on the full retain set under test-time transforms after the last step. It reads the stopping forget accuracy under statistics recomputed from 10,240 retain images (`-bn_calibrated_stop -bn_stop_recal_samples 10240`), ships the best epoch when the target is not met (`-restore_best_epoch`), and uses an epoch cap of 25.
- The retain-confusion distribution p_r is precomputed once from the frozen original model.
- Forward hooks are registered on `layer3`, `layer4`, and `avgpool` for feature-level diagnostics.
- Configuration files set $c = 1.0$; the CLI default is 5.0 (see above).
- All baselines are re-run within the same codebase under a unified MIA protocol, rather than quoted from their source papers.

Determinism and seeds. Every 10-class task is run with seeds $\{42, 123, 456\}$ and reported as mean $\pm$ std; CIFAR-100 uses a single seed per class. Seed variance is small (Appendix D). As an internal consistency check, the two ablation variants that are no-ops under the shipped configuration (`-rep_weight 0`, `-forget_gate 0`, both already 0) produce byte-identical metrics, confirming both that the runs are deterministic given a seed and that those components are genuinely inactive.

Known reproduction caveats. The floating-point exactness of the guarantee is precision-dependent (Remark 3): reference implementations should accumulate the projection inner products in float64. Wall-clock timings depend on the GPU and on how early dual stopping fires, so step counts rather than seconds are the portable quantity.